

Release 1 System Architecture and Technical Decision Record

1. Executive architecture summary

Release 1 should use a modular monolith consisting of one Next.js frontend, one FastAPI backend codebase deployed in two runtime roles, a web service and a background worker, one Supabase PostgreSQL database, private Supabase file storage, Supabase Auth, and one email provider. The backend remains the authority for all business rules. Slow document processing runs outside web requests through a PostgreSQL-backed job queue. Every processing attempt produces preserved candidate artifacts, while reviewer corrections and approved policy versions remain separate. Approved versions are immutable. The frontend receives processing updates through periodic polling, uses short-lived signed file access, and never has direct authority to approve policies, bypass limits, or alter audit history.

One-sentence architecture definition: A low-cost modular monolith that separates asynchronous AI processing, editable human review, and immutable approved insurance records while keeping one authoritative backend and one primary database.

Recommended high-level system shape

- Next.js frontend hosted on Vercel
- FastAPI web service hosted on Render
- FastAPI background worker hosted separately on Render
- Supabase PostgreSQL as the primary relational database
- Supabase private Storage for source PDFs and justified processing artifacts
- Supabase Auth for staff authentication
- PostgreSQL-backed background-job infrastructure
- Periodic frontend polling for Release 1 status updates
- Resend for background acknowledgement-email delivery
- Platform logs plus one dedicated error-tracking service
- Separate public-demo and future real-customer environments

The preferred platform direction fits Release 1. Vercel provides first-class Next.js deployment, Render provides dedicated background-worker services, Supabase combines PostgreSQL, Auth, and private Storage, and Resend supports duplicate-send protection through idempotency controls.

Why this is appropriate

This shape supports the product's real complexity—slow document processing, AI uncertainty, human approval, auditability, and demo isolation—without introducing distributed-system complexity. The web service and worker can scale independently while sharing the same business modules and database.

Complexity intentionally avoided

Release 1 does not require:

- microservices

- Kubernetes
- event streaming
- multiple operational databases
- real-time WebSockets
- multi-region hosting
- provider-routing infrastructure
- generalized workflow engines
- enterprise identity
- event sourcing
- a data warehouse
- a vector database

Status-model clarification

The approved specifications contain one small tension: a document may have an existing Approved outcome while a newly completed reprocessing attempt produces another result that Needs Review.

The smallest correction is to separate:

- the document's preserved approved disposition
- the current processing attempt
- the current review result
- the customer's current approved policy version

Therefore, an approved document can display:

Approved · New review available

The existing approved policy remains active while the new review result appears in the Review Queue. This does not change the approved UX; it makes the architecture capable of representing it consistently.

2. Architecture goals and non-goals

2.1 Goals

Reliable human-reviewed processing

The system must turn a supported declaration-page PDF into a reviewable candidate result without treating the AI output as approved business data.

Separation of trust

Source documents, extracted text, AI candidates, reviewer corrections, and approved policy versions must remain distinct.

Safe background processing

Document operations must continue after the browser closes and must not block ordinary web requests.

Traceability

Every important upload, processing attempt, correction, warning acknowledgement, approval, rejection, retry, email outcome, and demo reset must be explainable.

Controlled cost

The backend must enforce upload limits, page limits, retry limits, live-demo allowances, and processing quotas before paid work starts.

Public-demo reliability

Stored sample results must provide the complete review and approval experience even when AI or OCR services are unavailable.

Interview explainability

Every major component must have a clear purpose and ownership boundary.

Modular growth

Document processing, review, policy versions, audit, usage, and demo behavior must be able to evolve without splitting the system prematurely.

Document security

Insurance PDFs must remain private and accessible only through authorized, short-lived access.

Predictable recovery

Failures must preserve completed work and offer only recovery actions appropriate to the failure category.

2.2 Non-goals

Release 1 is not designed for:

- microservices
- real-time collaborative editing
- several insurance agencies
- telephony
- Voice AI
- advanced analytics
- enterprise-scale document throughput
- multi-region availability
- automatic provider failover
- arbitrary AI-provider routing
- custom extraction schemas
- a general agency management system
- quoting or binding insurance
- policy-status verification

- subscription billing
 - complex role management
 - public customer uploads
 - formal compliance certification
-

3. Architecture constraints inherited from approved documents

The architecture must support the following fixed behaviors.

Identity and access

- One agency context
- Authenticated staff users with the complete Release 1 workflow
- Controlled demo users with restricted behavior
- No custom role matrix
- No invitation or workspace-management workflow

Upload

- One PDF per upload
- Maximum 15 MB
- Maximum 10 pages
- One policy per file
- Private document storage
- Duplicate detection before another paid attempt
- No arbitrary local uploads in public demo mode

Processing

- Asynchronous execution
- Direct PDF text first
- OCR only when required
- One production AI provider
- Structured candidate extraction
- Source-page evidence when available
- Clear failure categories
- Two user-triggered technical retries after the initial attempt
- Intentional reprocessing
- Stored-result replay
- Live processing that can be disabled

Review

- Candidate information remains untrusted
- Saved reviewer corrections
- Warning acknowledgement
- Mark Not Present
- Reviewer-added and reviewer-removed vehicles and drivers
- Approval-readiness calculation
- Stale-data protection

- Desktop two-panel review
- Independently loading PDF and review panels

Approval and policies

- Approval as a deliberate business transaction
- Immutable approved policy versions
- One selected current uploaded policy version
- Preserved historical policy versions
- Explicit replacement of the current version
- Correction Review without an AI call
- Optional explicit customer-contact-address update
- Repeated approval protection

Audit and communication

- Business-understandable audit history
- Reviewer-triggered acknowledgement email
- Email outcome independent of approval
- Duplicate-send protection

Demo

- Synthetic template data
- Disposable, isolated demo changes
- Stored sample results by default
- Optional restricted live processing
- Simulated email
- Reset from Demo Overview or account utility area
- Provider-outage resilience

Reliability

- Safe duplicate submissions
- Safe retry and reprocessing behavior
- Long-running-work detection
- Processing-status refresh
- Browser closure must not cancel processing
- Web and worker deployments must remain compatible

4. System context

4.1 Main participants and systems

Staff user

Uses the product to manage customers, upload documents, review extracted information, approve policies, and send acknowledgements.

Controlled demo user

Uses synthetic, disposable data and curated documents. Cannot bypass live-processing limits or send arbitrary emails.

Browser

Runs the frontend interface and PDF viewer. It holds temporary interface state but is not trusted to enforce business rules.

Frontend application

Presents product state, gathers user decisions, protects unsaved local changes, and communicates with the backend.

Backend application

Acts as the business authority. It authenticates requests, authorizes actions, enforces rules, controls signed file access, creates jobs, approves policy versions, and records audit events.

Relational database

Stores authoritative business state, background-job state, processing-attempt metadata, review decisions, approved versions, usage, audit, and demo-session state.

Private document storage

Stores original PDFs and justified processing artifacts.

Background worker

Performs document reading, OCR, AI extraction, validation, evidence preparation, replay processing, and email delivery.

PDF-reading capability

Extracts direct text and document metadata without using AI.

OCR capability

Produces text from scanned pages only when direct text is inadequate.

AI extraction provider

Produces structured candidate values from prepared document content.

Email provider

Sends the fixed acknowledgement message for eligible approved documents.

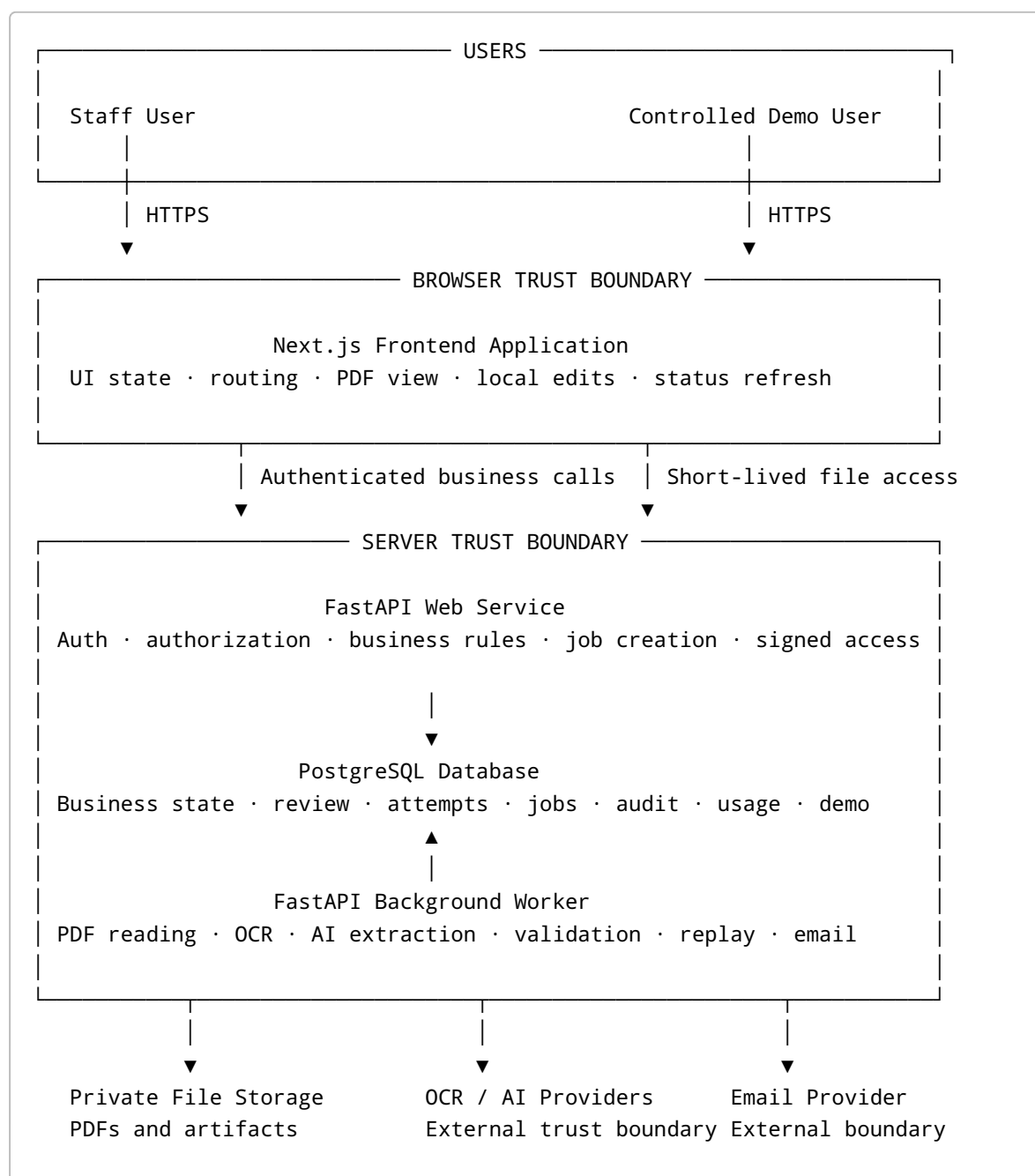
Monitoring tools

Collect operational logs, errors, health information, and reference identifiers.

4.2 Trust boundaries

- The browser is untrusted for business enforcement.
- The backend is the business-policy authority.
- The worker is trusted as an internal runtime but still validates job ownership and current state.
- External providers receive only the content necessary for their capability.
- Private documents are never placed in a public storage location.
- Demo and future customer environments remain separated.

4.3 System-context diagram



5. Recommended deployment topology

5.1 Recommendation

Keep the preferred deployment split:

- Next.js frontend on Vercel
- FastAPI web service on Render
- separate FastAPI worker on Render
- Supabase PostgreSQL, Storage, and Auth
- Resend for email

5.2 Why the split fits

Vercel

Owns frontend builds, static delivery, Next.js rendering, and frontend environment configuration. Vercel's official documentation directly supports Next.js deployments and environment-specific configuration.

Render web service

Runs the public FastAPI backend and health checks. It should remain stateless between requests.

Render worker

Runs long-lived background processing. Render has a dedicated background-worker service type intended to move long-running work out of the request path.

Supabase

Provides the primary PostgreSQL database, private object storage, and authentication. Supabase Storage supports access policies and time-limited signed URLs, while Supabase Auth issues signed user-session tokens that can be verified by backend services.

Resend

Handles the one approved transactional email workflow and supports provider-level idempotency for duplicate-send protection.

5.3 Communication

- Browser communicates with FastAPI over HTTPS for business operations.
- Browser communicates with Supabase Auth for sign-in and session refresh.
- FastAPI verifies the authenticated identity and backend membership.
- FastAPI and worker communicate indirectly through PostgreSQL job state.

- FastAPI and worker access Supabase PostgreSQL and private Storage using server-only credentials.
- Browser accesses a PDF only through a short-lived URL issued after backend authorization.
- Worker communicates with PDF, OCR, AI, and email capabilities.
- Frontend never communicates directly with OCR, AI, or email providers.

5.4 Direct-to-storage upload

The preferred upload path is:

1. Backend authorizes an upload for a selected customer.
2. Backend issues restricted, short-lived upload permission.
3. Browser transfers the PDF directly to private Storage.
4. Backend confirms the completed upload and creates processing work.

This avoids routing large PDF bytes through the FastAPI web service while keeping the backend in control.

5.5 Independent scaling

Can scale independently:

- frontend delivery
- FastAPI web requests
- background workers
- PostgreSQL capacity
- storage capacity

Must remain version-compatible:

- web service and worker
- database migrations and application releases
- stored-result definitions and review interpretation

5.6 Components that must not communicate directly

- Browser must not call AI or OCR providers.
- Browser must not use server-level Supabase credentials.
- Browser must not create approved policy versions directly.
- Browser must not write audit events directly.
- Worker must not make user authorization decisions from job data alone.
- Frontend must not determine whether retry or live-processing limits have been reached.
- Email provider must not determine business eligibility.

5.7 Secret storage

- Vercel stores frontend server-side configuration.
- Render stores backend and worker secrets.
- Only public or publishable authentication configuration may reach the browser.
- Database service credentials, storage service credentials, AI keys, OCR keys, and email keys remain server-only.

Vercel and Render both support environment-based secret configuration rather than committing secrets to source control.

5.8 Platform outage effects

Vercel unavailable

The interface cannot load, but background processing may continue.

Render web unavailable

New business actions and status queries stop. Existing worker processing may continue.

Render worker unavailable

Uploads may be accepted only if the system can clearly indicate that processing is waiting. No job should be lost.

Supabase unavailable

Business actions and processing stop safely because authoritative state and private files are unavailable.

Resend unavailable

Approval remains valid. Email becomes Failed or Pending Retry.

5.9 Public-demo reliability

Render's free web services currently spin down after inactivity and can take around a minute to become available again. That behavior is acceptable for development but weak for a polished public demo. The public demo should move to always-on paid web and worker capacity when cold starts begin damaging Loom recordings or client demonstrations.

5.10 Simpler consolidated alternative

A simpler deployment would place:

- Next.js frontend
- FastAPI backend
- worker
- PostgreSQL

on one general-purpose server.

Benefits

- fewer platforms
- simpler networking
- one bill
- complete runtime control

Limitations

- more server administration
- manual SSL, deploy, backup, process, and scaling responsibility
- weaker preview deployment experience
- harder separation of web and worker capacity
- less portfolio value from modern managed deployment

5.11 Decision

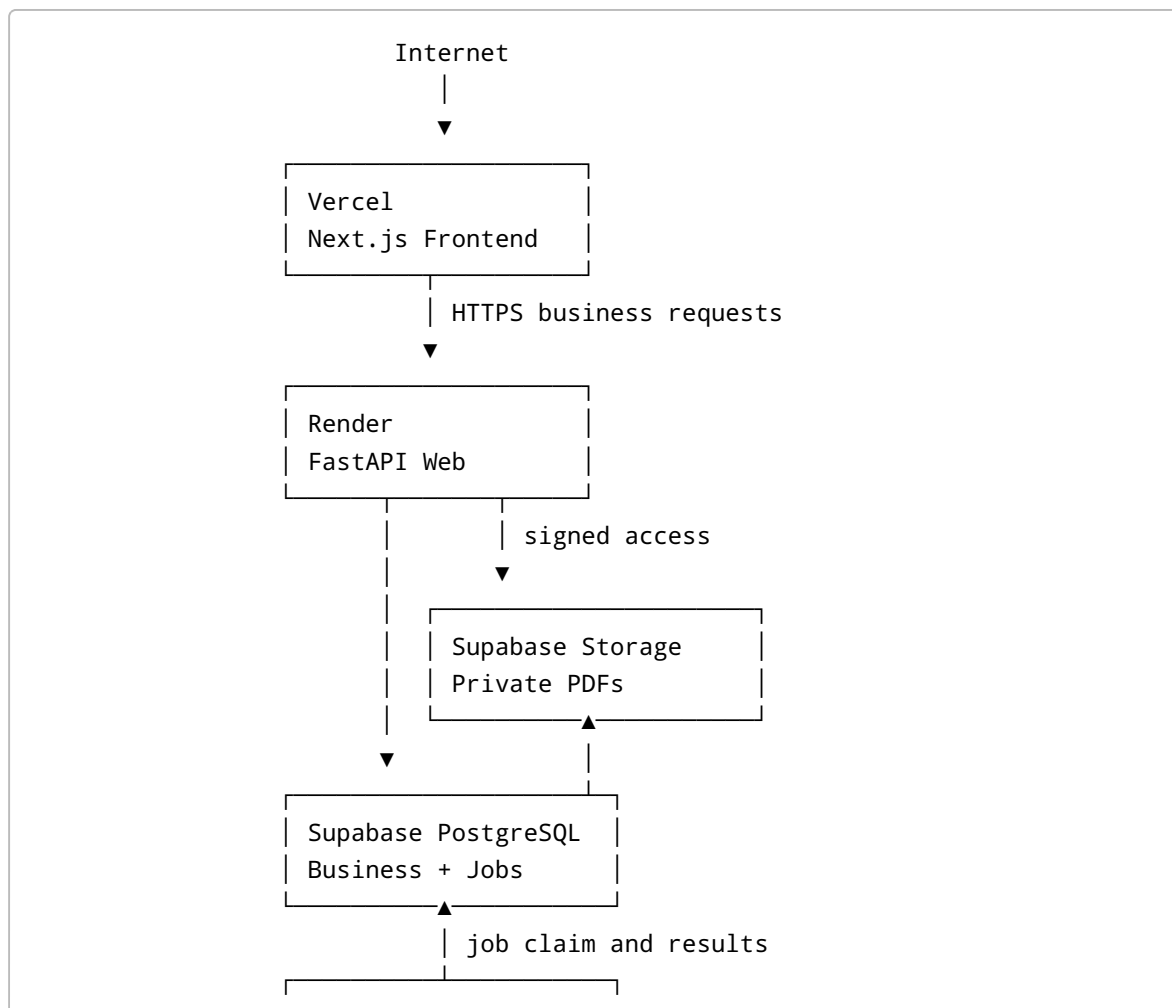
Keep Vercel, Render, and Supabase.

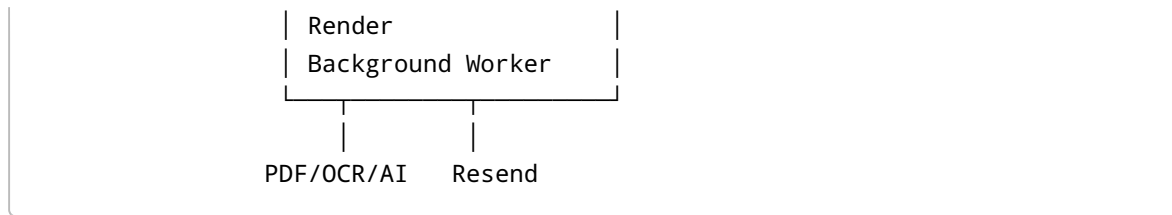
The accepted trade-off is operating three platforms instead of one. The benefit is less infrastructure administration and clearer independent deployment responsibilities.

Revisit consolidation if:

- cross-platform latency becomes material
- the platform count creates more operational work than it saves
- costs exceed a small consolidated server
- a real customer requires a single cloud boundary

5.12 Deployment diagram





6. Frontend architectural responsibilities

6.1 What belongs in the frontend

Application shell

- navigation
- account utility area
- Demo indicator
- processing-limit indicator
- breadcrumbs
- responsive layout

Routing

- protected page navigation
- return-to-original-page behavior after sign-in
- current search, filtering, sorting, and tab state
- review and policy-history routes

Authentication state

- sign-in UI
- session-aware routing
- sign-out
- session-expired presentation
- token refresh integration

Server-state fetching and caching

- customer lists
- documents
- processing status
- review data
- approved policies
- audit history
- usage

React Query remains suitable for server-state caching and controlled refresh.

Local unsaved edits

- currently edited field value
- unsaved customer form

- unsaved warning note
- unsaved email-preview state
- navigation protection

Review interaction

- field expansion
- source-page selection
- panel resizing
- keyboard focus
- warning navigation
- readiness presentation
- approval drawer

PDF viewing

- page rendering
- zoom
- rotation
- evidence highlighting where available
- fallback state

Accessibility

- semantic page structure
- focus control
- keyboard behavior
- status announcements
- color-independent warning presentation

Processing refresh

- polling while relevant documents remain active
- reduced polling when the tab is in the background
- refresh on tab focus
- completion notification

Demo guidance

- stored-result labels
- guided workflow
- restricted-action explanations
- Reset Demo entry

Error presentation

- user-facing categories from the backend
- retained local edits after recoverable failure
- reference identifiers

6.2 What must not be trusted to the frontend

The frontend must not authoritatively decide:

- whether a user may access a customer
- whether a document belongs to the agency
- whether a PDF is accepted
- whether processing limits permit work
- whether retry limits remain
- whether approval readiness is satisfied
- whether a policy may become current
- whether an audit event is valid
- whether an email is eligible
- whether demo live processing is allowed
- whether a signed file URL should be issued
- whether a repeated submission is a duplicate

Frontend checks improve UX but are not security or business enforcement.

6.3 State separation

Local interface state

Temporary and browser-owned:

- open tab
- panel size
- expanded field
- unsaved input
- PDF zoom
- active filter

Loss is inconvenient but must not corrupt business records.

Persisted review state

Server-owned but editable before approval:

- saved reviewer correction
- Mark Not Present
- acknowledgement
- added or removed list item
- reviewer note

Server-owned business state

Authoritative:

- customer
- document lifecycle
- processing attempt
- usage allowance

- approved policy version
 - current-policy selection
 - audit history
 - demo-session restrictions
-

7. Backend architectural responsibilities

7.1 Identity enforcement

The backend verifies authenticated identities and rejects invalid or expired sessions.

7.2 Authorization

The backend determines whether the user may access:

- the agency context
- a customer
- a document
- a review
- a policy version
- a demo session
- a signed file URL

7.3 Customer operations

The backend controls:

- customer creation
- duplicate-customer warnings
- edits to contact information
- customer search
- customer-policy relationships

7.4 Document intake

The backend controls:

- upload authorization
- selected customer
- file-size and page limits
- file-type validation
- duplicate detection
- final upload acceptance
- job creation

7.5 Processing orchestration

The backend:

- creates initial attempts

- reserves usage allowance
- prevents simultaneous duplicate attempts
- creates background jobs
- exposes status
- classifies final failures

7.6 Review management

The backend owns:

- editable review state
- reviewer corrections
- warning resolution
- Mark Not Present
- reviewer-added and removed items
- readiness calculation
- freshness version
- stale-save rejection

7.7 Approval rules

The backend rechecks:

- document status
- review freshness
- required fields
- warnings
- customer
- active policy
- address-update decision
- repeated approval

7.8 Policy versions

The backend creates immutable approved versions and controls Current or Historical selection.

7.9 Retry enforcement

The backend enforces:

- retry eligibility
- maximum two user-triggered technical retries
- one active attempt
- paid-processing confirmation
- usage allowance

7.10 Duplicate protection

The backend protects:

- customer submission
- upload acceptance

- processing
- corrections
- approval
- rejection
- retry
- reprocessing
- email
- demo reset

7.11 Usage limits

The backend reserves and records usage before paid work starts.

7.12 Audit creation

The backend ensures business events are created with the corresponding business action.

7.13 Email eligibility

The backend verifies:

- approved policy exists
- valid customer email exists
- recipient is permitted
- duplicate-send state
- demo behavior

7.14 Demo rules

The backend controls:

- demo identity
- demo-session partition
- sample access
- live allowance
- email simulation
- reset
- data expiration

7.15 Signed document access

The backend authorizes each file request before issuing short-lived access.

7.16 Stale-data checks

The backend rejects final actions based on an outdated:

- review result
- customer version
- active policy
- processing attempt

- approval state

7.17 Transactional business actions

The following must succeed or fail as one logical business operation:

Customer creation

- create customer
- record audit event
- register duplicate override when applicable

Upload acceptance

- confirm file metadata
- associate customer
- create document
- create initial attempt
- reserve usage
- create job
- record audit event

If the job cannot be registered safely, the document must not appear as actively processing without recoverable queued work.

Saved correction

- verify freshness
- save correction
- update review readiness
- create audit event

Approval

- verify readiness and freshness
- create immutable policy version
- select Current or Historical
- demote previous Current when applicable
- optionally update contact address
- mark review approved
- update document presentation state
- record audit events

Rejection

- verify freshness
- record rejection
- preserve active policy
- record audit event

Demo reset

- clear disposable state

- recreate seed state
 - record reset outcome within demo context
-

8. Modular-monolith boundaries

8.1 Identity and Access

Responsibility

User identity, sessions, staff membership, demo identity, and access decisions.

Conceptual state owned

- user identity linkage
- agency membership
- demo-session identity
- access context

Actions controlled

- access checks
- actor resolution
- demo-versus-staff classification

Consumes

Authentication-provider session claims.

Must not modify

Customers, reviews, policy versions, usage, or audit history directly.

Why separate

Identity rules affect every module and must not be duplicated.

8.2 Customers

Responsibility

Customer contact information and customer lookup.

State owned

- full name
- email
- phone
- contact address
- customer duplicate indicators

Actions controlled

- create
- edit
- search
- contact-address update

Consumes

Approved mailing address only when the reviewer explicitly chooses an update.

Must not modify

Extracted candidate data or approved policy history.

Why separate

Customer contact data has a different trust level and lifecycle from document-derived policy data.

8.3 Documents**Responsibility**

Source-document identity, customer association, storage reference, primary disposition, and upload rules.

State owned

- source file identity
- upload metadata
- customer association
- duplicate relationship
- primary document disposition

Actions controlled

- upload acceptance
- source access
- replacement relationship
- document-level status presentation

Consumes

Processing and review outcomes.

Must not modify

Candidate extraction content or approved policy values directly.

8.4 Processing

Responsibility

Processing attempts, job orchestration, stages, failures, retries, reprocessing, and replay.

State owned

- processing attempt
- active attempt
- stage
- timing
- failure category
- retry count
- processing origin

Actions controlled

- queue work
- claim work
- start attempt
- complete attempt
- fail attempt
- retry
- reprocess
- replay

Consumes

Source document and usage authorization.

Must not modify

Approved policy versions.

8.5 Extraction Results

Responsibility

Preserved output from document reading, OCR, AI extraction, normalization, validation, and evidence preparation.

State owned

- extracted text
- OCR output
- raw provider response reference
- normalized candidate
- validation result
- source evidence

Actions controlled

- publish a successful candidate result
- expose evidence to Review

Consumes

Processing artifacts.

Must not modify

Customer contact data, review corrections, or approved versions.

8.6 Review**Responsibility**

Human-editable interpretation of one reviewable candidate or approved baseline.

State owned

- review baseline
- corrections
- acknowledgements
- Not Present decisions
- added and removed items
- readiness
- review freshness
- rejection

Actions controlled

- save correction
- resolve warning
- calculate readiness
- request approval
- reject
- start Correction Review

Consumes

Candidate values, validation, evidence, customer context, and existing approved policy.

Must not modify

Approved policy versions directly; it requests Policy Versions to approve.

8.7 Policy Versions**Responsibility**

Immutable approved policy records and current-versus-historical selection.

State owned

- approved policy version
- version relationship
- Current or Historical designation
- approval metadata

Actions controlled

- create approved version
- replace Current
- approve Historical
- create corrected version

Consumes

Final Review state and explicit address-update decision.

Must not modify

Source extraction artifacts or previous approved version content.

8.8 Audit

Responsibility

Append-only business audit events.

State owned

- actor
- action
- timestamp
- related business objects
- approved previous/new values where appropriate

Actions controlled

- append business event
- present filtered business history

Consumes

Business-action facts from every module.

Must not modify

The business action it records.

8.9 Usage and Limits

Responsibility

Processing allowance, usage reservation, actual usage, estimates, and demo limits.

State owned

- configured limits
- reserved usage
- finalized usage
- provider-reported quantities
- cost estimates

Actions controlled

- authorize paid work
- reject work over limit
- record usage outcome

Consumes

Document pages and provider usage results.

Must not modify

Processing results or policy data.

8.10 Email Acknowledgement

Responsibility

Post-approval email eligibility, delivery state, duplicate protection, and simulation.

State owned

- send request
- recipient
- provider outcome
- resend state
- simulation status

Actions controlled

- request send
- complete send
- fail send
- resend

Consumes

Approved policy, customer email, staff/demo context.

Must not modify

Approval or customer policy state.

8.11 Demo Environment

Responsibility

Synthetic templates, disposable sessions, sample access, reset, and demo restrictions.

State owned

- demo-session identity
- expiration
- seeded object references
- reset status
- live allowance

Actions controlled

- start session
- seed session
- reset session
- expire session
- authorize sample replay or limited live processing

Consumes

Immutable template definitions and sample artifacts.

Must not modify

Real staff data.

8.12 Shared Platform Services

Responsibility

Provider boundaries, clocks, reference identifiers, storage access, logging, configuration, and common transaction support.

Must not own

Insurance business decisions.

8.13 Internal communication principle

Modules communicate through explicit business operations and stable internal contracts.

They should not:

- reach into another module's state and change it directly
- duplicate readiness or authorization rules
- infer approval from document status alone
- construct audit events independently of business context

The monolith may share one process and database while maintaining ownership boundaries in design and review.

9. Synchronous versus background work

9.1 Immediate request-response work

Suitable for short, deterministic operations:

- session validation
- customer creation and edit
- duplicate-customer check
- upload authorization
- cheap file metadata checks
- review correction
- warning acknowledgement
- Mark Not Present
- review readiness query
- approval
- rejection
- usage lookup
- signed file-access request
- demo reset request registration

9.2 Background work

Required for uncertain or slow operations:

- deeper PDF inspection
- direct text extraction
- OCR
- AI extraction
- evidence preparation
- full candidate validation
- stored-result replay timing
- email delivery
- cleanup of temporary artifacts
- expiration of old demo sessions

9.3 User-confirmed business transactions

These are synchronous because the user must receive a definitive outcome:

- approval
- rejection
- intentional duplicate processing authorization
- technical-retry request
- reprocessing request
- current-policy replacement decision
- contact-address update decision

The expensive work triggered by retry or reprocessing remains background.

9.4 Deferred notifications

- processing-completion notification
- email-delivery result
- delayed-processing classification
- cleanup warnings

9.5 Why processing must be background work

Document processing may involve:

- several PDF pages
- OCR
- external AI calls
- retries
- provider timeouts
- variable latency

Keeping it inside a web request would create:

- request timeouts
- poor browser experience
- duplicate submissions
- unclear recovery
- web-service resource pressure

9.6 Immediate confirmation

After upload acceptance, the user receives:

- accepted file
- associated customer
- created processing attempt
- Processing status
- permission to leave the page

9.7 Browser closure

The worker claims work from server-owned job state. The browser is not part of processing execution.

9.8 Status updates

The worker records durable stage changes. The frontend polls the backend.

9.9 Failed work

A final failure:

- marks the attempt failed
- updates user-visible document work state
- records a plain-language failure category
- finalizes usage
- creates audit and diagnostic context

9.10 Duplicate-job prevention

Before starting work, the worker verifies:

- job is still active
- attempt is still claimable
- no other worker holds a valid claim
- the document has no conflicting active attempt

9.11 Long-running-work detection

Each attempt records durable progress timing.

A periodic check identifies attempts that:

- have not started within the accepted queue period
- have not reported progress within the expected stage window
- exceeded the maximum processing duration

10. Background-job architecture decision

10.1 Option A: Separate worker with PostgreSQL-backed jobs

Advantages

- uses the existing database
- no additional paid queue platform
- durable job and business state in one transactional system
- good visibility for low Release 1 volume
- straightforward local development
- supports deduplication and attempt ownership
- sufficient for document-processing jobs measured in seconds or minutes

Limitations

- job polling adds database load
- database must handle both business and job traffic
- delayed scheduling and advanced queue features require deliberate design
- very high throughput would justify dedicated queue infrastructure

10.2 Option B: Redis-backed jobs

Advantages

- mature queue semantics
- efficient job claiming
- good worker scaling
- natural retry and scheduling support

Limitations

- another paid stateful service
- another credential and outage dependency
- queue state and business state cannot be committed together
- persistence settings matter
- more local-development setup

Render supports Key Value infrastructure, but persistence and availability characteristics vary by plan, adding another operational decision.

10.3 Option C: Managed workflow service

Advantages

- built-in retries
- durable workflows
- orchestration visibility
- useful for complex, multi-day workflows

Limitations

- additional provider
- higher conceptual complexity
- unnecessary for a short linear Release 1 pipeline
- can encourage provider-specific workflow coupling

10.4 Option D: Long work inside the web process

Advantages

- smallest initial code footprint
- no separate worker

Limitations

- request timeouts

- browser dependency
- web-process resource contention
- weak retry guarantees
- duplicate risk
- unsafe deploy interruption
- poor scaling

This option is rejected.

10.5 Recommendation

Use:

A separate Render worker with a PostgreSQL-backed durable job queue.

10.6 Why this is preferred

It addresses the actual Release 1 requirement—durable, low-volume asynchronous work—without adding Redis or a workflow platform.

10.7 Accepted trade-off

PostgreSQL carries both business and job workload.

10.8 Required characteristics

The queue must conceptually support:

- durable pending jobs
- one active claim
- claim expiration
- retry scheduling
- job deduplication
- attempt association
- failed-job visibility
- graceful worker restart
- job-version compatibility

10.9 Revisit triggers

Move to Redis-backed or managed job infrastructure when:

- PostgreSQL job polling materially affects business queries
- worker count grows significantly
- document throughput becomes sustained rather than occasional
- priority queues become necessary
- workflows span many hours or days
- scheduled callbacks become common
- job fan-out becomes complex

11. Document lifecycle ownership

11.1 Ownership

The Documents module owns the document's stable user-facing disposition.

The Processing module owns attempt execution.

The Review module owns reviewability and review completion.

The Policy Versions module owns approved business results.

11.2 Main displayed states

Processing

A current attempt is pending or running and no reviewable result is currently selected.

Needs Review

A current review result exists and has not been approved or rejected.

Approved

At least one review result from this document has produced an approved policy version.

Rejected

The selected review result was rejected and no active unreviewed result remains.

Failed

The latest relevant attempt failed without creating a reviewable result.

11.3 Composite states

To support approved reprocessing:

- Approved · Reprocessing
- Approved · New review available
- Approved · Reprocessing failed

The Approved outcome remains preserved.

11.4 Secondary descriptions

Secondary descriptions belong to the active attempt:

- Waiting to process
- Checking file
- Reading document text

- Reading scanned pages
- Extracting policy information
- Checking extracted information
- Taking longer than expected
- Reprocessing

11.5 Allowed-transition principle

Transitions are controlled by module operations, not arbitrary status updates.

Examples:

- accepted upload → Processing
- successful attempt → Needs Review
- review approval → Approved
- review rejection → Rejected
- failed attempt → Failed
- Approved + new attempt → Approved · Reprocessing
- Approved + successful new attempt → Approved · New review available

11.6 Invalid transitions

Reject:

- Failed directly to Approved
- Processing directly to Approved
- Rejected directly to Approved without a new review result
- Approved policy being erased by failed reprocessing
- Needs Review being replaced by another attempt without deliberate reprocessing

11.7 Transition history

Business-significant transitions create audit events.

Fine-grained processing stages belong to attempt diagnostics and selected user-facing activity.

11.8 Delayed processing

Delayed is not a final state. It is an active-attempt condition.

11.9 Failed reprocessing

The new attempt fails, while:

- previous approval remains
- failure is visible
- attempt history is preserved
- customer policy remains unchanged

11.10 Preserving outcomes

No new attempt overwrites:

- previous failed attempts
 - rejected reviews
 - approved versions
 - corrections associated with previous reviews
-

12. Processing-attempt architecture

12.1 Conceptual relationship

A Document may have several Processing Attempts.

Each attempt may produce:

- direct-text result
- OCR result
- AI extraction result
- normalized candidate
- validation result
- source evidence
- reviewable result

12.2 Why attempts are preserved

Overwriting attempts would make it impossible to explain:

- why results changed
- which provider call failed
- which configuration produced an approval
- whether a retry consumed usage
- whether reprocessing improved extraction
- which result the reviewer corrected

12.3 Initial attempt

Created after upload acceptance and quota reservation.

12.4 Automatic limited recovery

A stage may perform limited internal recovery for transient errors.

This does not consume one of the two user-triggered retries, but it remains visible diagnostically.

12.5 User-triggered Technical Retry

Creates a new attempt linked to the failed attempt.

12.6 Intentional Reprocessing

Creates a new attempt even when a successful or approved result exists.

12.7 Stored demo result

Creates an attempt identified as Stored Result.

It uses prepared extraction artifacts rather than external PDF, OCR, or AI execution.

12.8 Replayed processing

The attempt progresses through real durable workflow states for demonstration, but expensive provider stages are replaced by approved stored artifacts.

12.9 Failed attempt

Preserves:

- completed stages
- failure category
- usage
- timing
- diagnostic reference

12.10 Successful attempt

Publishes a normalized candidate and evidence that can be selected for review.

12.11 Active attempt

A document should have no more than one paid or replay-processing attempt active at once.

12.12 Latest reviewable attempt

A document may have several successful attempts, but only one is selected as the current candidate under review.

Previous candidates remain preserved.

13. Separation of information trust levels

13.1 Original source document

Nature

Immutable source supplied by the user.

Trust

Evidence source, but not automatically verified as current or authentic.

Retention

Preserved while the document record exists.

May update customer record

No.

13.2 Extracted direct text

Nature

Machine-produced representation of source PDF text.

Trust

Untrusted processing artifact.

Retention

Preserved per successful attempt when needed for reproducibility.

May update customer record

No.

13.3 OCR text

Nature

Machine-read text from scanned pages.

Trust

Untrusted and potentially error-prone.

Retention

Preserved per attempt when useful for evidence and diagnosis.

May update customer record

No.

13.4 Raw provider response

Nature

Provider-specific output.

Trust

Untrusted.

Retention

Restricted and time-limited unless needed for reproducible evaluation.

Sensitivity

May contain personal information.

May update customer record

Never.

13.5 Normalized candidate values

Nature

Provider output converted into the product's supported field representation.

Trust

Unapproved candidate.

Retention

Immutable per attempt.

May update customer record

No.

13.6 Validation and evidence signals

Nature

Missing, conflict, format, OCR, and source-support analysis.

Trust

Decision support, not proof.

Retention

Immutable with the candidate version.

13.7 Reviewer corrections

Nature

Human-edited proposed values.

Trust

More authoritative than the AI candidate, but not approved until the approval transaction.

Editable

Yes, before approval.

13.8 Warning acknowledgements

Nature

Human decision that a known warning is acceptable.

Editable

Before approval.

Retention

Preserved with the review and approved version context.

13.9 Approved policy version

Nature

Immutable business-approved interpretation.

Trust

Trusted internally as staff-reviewed data, but not carrier verification.

Editable

No.

May update current policy summary

Yes.

13.10 Customer contact information

Nature

Staff-maintained customer contact data.

Trust

Independent business data.

Editable

Yes.

Relationship to policy address

Never silently replaced.

13.11 Why layers must not be collapsed

Collapsing them would:

- erase extraction history
- hide reviewer intervention
- permit AI output to become business truth
- make retries overwrite evidence
- make correction reviews impossible to explain
- weaken auditability
- complicate model evaluation

14. Review-state architecture

14.1 Saved corrections

Stored as deliberate changes relative to a fixed baseline:

- candidate baseline
- or approved-version baseline for Correction Review

14.2 Unsaved local edits

Remain in browser state until submitted.

The backend does not consider them part of readiness.

14.3 Mark Not Present

A saved reviewer decision distinct from an empty field.

14.4 Warning acknowledgement

A saved resolution linked to:

- warning
- reviewer
- explanation when required
- baseline version

14.5 Added vehicles and drivers

Represent reviewer-proposed items relative to the baseline.

14.6 Removed vehicles and drivers

The baseline item remains preserved while the review excludes it from the proposed final result.

14.7 Readiness calculation

The backend calculates readiness using:

- required field completion
- unresolved warnings
- allowed acknowledgements
- saved corrections
- customer-identity confirmation
- current review freshness
- current processing result
- document status

14.8 Stale-data detection

Each editable review state has a freshness identity.

Saving or approving against an older identity is rejected.

14.9 Correction history

Material saved corrections create audit events.

The review also retains enough structured history to show baseline and final proposed value.

14.10 Multiple users

Several users may open a review.

The system does not hard lock it.

It prevents stale saves and final actions from overwriting newer work.

14.11 Editable review result

Based on one fixed candidate attempt.

May be corrected and acknowledged.

14.12 Approved version

Immutable output produced by approval.

14.13 Correction Review

New editable review based on an approved policy version.

It does not reuse or rerun AI processing.

15. Approval and policy-version architecture

15.1 Approval sequence

The backend must:

1. Verify identity and access.
2. Verify document and customer still exist.
3. Verify the review remains current.
4. Verify its baseline attempt or approved version has not changed.
5. Recalculate readiness.
6. Verify no unresolved blocking warning remains.
7. Verify the active policy context.
8. Create a new immutable approved version.
9. Select it as Current or Historical.
10. Preserve or replace the previous Current selection.
11. Apply the optional contact-address update only when explicitly selected.
12. Mark the review approved.
13. append audit events.
14. Return one definitive result.

15.2 Effects that must succeed together

- approved version creation
- approval identity and timestamp
- Current/Historical designation
- previous Current demotion when applicable
- review finalization
- document presentation update
- explicit contact-address update
- core approval audit events

If one fails, none should be presented as completed.

15.3 Immutable version

The final values are copied into a preserved approved representation.

They are not a live view of editable review state.

15.4 Current versus Historical

Current is a selection relationship, not a mutation of the previous version's values.

15.5 Replacement

The previous Current version becomes Historical but remains unchanged.

15.6 Repeated approval

A repeated request returns the already completed approval when it represents the same business action.

It must not create another version.

15.7 Failure rollback

A failed approval leaves:

- review editable
- previous Current unchanged
- no partial new policy version
- no false approval audit event

15.8 Why approval is not an ordinary update

An ordinary mutable policy record would:

- hide historical values
- lose reviewer evidence
- create race conditions
- make correction and renewal behavior unclear
- weaken interview explainability
- make repeated approval dangerous

16. Correction Review architecture

16.1 Baseline

Correction Review begins from one immutable approved version.

16.2 Proposed corrections

The review stores only the proposed differences and review decisions relative to the baseline.

16.3 Original approved version

Remains unchanged and Current until a new corrected version is approved.

16.4 New corrected version

Approval creates a new immutable version linked as a correction of the earlier version.

16.5 Current replacement

The new corrected version normally becomes Current.

The previous version becomes Historical.

16.6 Audit

Records:

- Correction Review started
- values changed
- review abandoned when material
- corrected version approved
- previous version replaced

16.7 Cancellation

Cancellation or abandonment deletes or closes only the unapproved correction work.

It does not alter approved policy data.

16.8 Difference from reprocessing

Correction Review

- no PDF, OCR, or AI call
- approved values are the baseline
- human-only correction
- no paid document-processing cost

Reprocessing

- source document is processed again
- produces another AI candidate
- may consume paid usage
- must be reviewed independently

17. Retry and reprocessing architecture

17.1 Automatic internal recovery

Initiated by the worker for a narrowly defined transient failure.

It must be limited and must not create uncontrolled provider calls.

17.2 Technical Retry

Initiated by staff after a retryable final failure.

Rules:

- maximum two user-triggered retries
- backend enforcement
- paid-processing confirmation
- one active attempt
- prior attempts preserved
- usage recorded

17.3 Intentional Reprocessing

Initiated by staff for a completed result.

Rules:

- allowed for eligible documents
- explicit paid-processing warning
- existing approval remains active
- new result requires review
- no retry-limit count, but governed by general usage and reasonable reprocessing controls

17.4 Correction Review

Human-only correction from an approved baseline.

17.5 Replacement upload

Creates a new document when the source PDF changes or is unusable.

17.6 Protection of approved data

None of these actions changes approved policy data until another approval succeeds.

17.7 Simultaneous attempts

The Processing module rejects a new attempt when another active attempt exists for the same document.

17.8 Failure

Failure finalizes only the new attempt.

17.9 Usage

Each attempt distinguishes:

- live
- stored
- internal recovery
- technical retry
- reprocessing

17.10 Audit

User-triggered actions create business audit events.

Low-level internal recovery belongs primarily to diagnostic logs and attempt history.

18. Idempotency and duplicate-action safety

18.1 File upload acceptance

Risk

Connection retry creates several document records.

Safe outcome

One accepted upload and one initial attempt.

Enforced by

Documents and Processing modules.

Repeated-action UX

User sees the existing accepted document.

18.2 Duplicate document processing

Risk

Same PDF starts several paid attempts.

Safe outcome

Duplicate warning or return of existing active attempt.

Enforced by

Documents, Processing, and Usage modules.

18.3 Customer creation**Risk**

Double click or network retry creates duplicate customers.

Safe outcome

One customer for the same creation submission.

Enforced by

Customers module.

UX

User lands on the created customer.

18.4 Correction saving**Risk**

Repeated save creates duplicate history or overwrites newer correction.

Safe outcome

One saved correction against the expected review version.

Enforced by

Review module.

18.5 Approval**Risk**

Several approved versions or repeated Current replacement.

Safe outcome

One approval and one immutable version.

Enforced by

Review and Policy Versions modules transactionally.

18.6 Rejection

Risk

Repeated rejection events or conflict with approval.

Safe outcome

One final review outcome.

Enforced by

Review module.

18.7 Technical Retry

Risk

Several workers process separate retries.

Safe outcome

One new retry attempt.

Enforced by

Processing and Usage modules.

18.8 Reprocessing

Risk

Repeated paid reprocessing.

Safe outcome

Return existing active reprocessing attempt.

18.9 Acknowledgement email

Risk

Customer receives duplicate messages.

Safe outcome

One provider send for one confirmed send action.

Enforced by

Email module plus provider-level idempotency where available. Resend supports idempotency controls specifically intended to prevent duplicate delivery.

18.10 Demo reset

Risk

Repeated reset leaves partial seed data.

Safe outcome

One complete known demo state.

Enforced by

Demo Environment module transactionally or through a repeat-safe reset workflow.

19. Concurrency and stale-data strategy

19.1 Compared approaches

Last-write-wins

Simplest technically.

Rejected because it can erase reviewer work or approve outdated information.

Hard locking

Prevents concurrent editing.

Rejected for Release 1 because:

- abandoned locks require recovery
- browser closure complicates ownership
- assignment behavior was explicitly excluded
- users may still need read access

Optimistic concurrency

Allows several users to view and edit, but checks freshness during save and final actions.

19.2 Recommendation

Use optimistic concurrency with freshness checks.

19.3 Why it fits

It prevents stale overwrites without introducing assignment and lock management.

19.4 Checks before correction save

- review is still editable
- baseline is unchanged
- expected review version matches
- document was not approved or rejected elsewhere

19.5 Checks before approval or rejection

- current review identity
- baseline attempt identity
- customer freshness
- Current policy context
- review outcome
- readiness
- user session
- active attempt state

19.6 Two users reviewing

Both may view.

A stale save is rejected and the user is shown the current result.

19.7 Approval while another edits

Approval succeeds only for the current saved review version.

The second user's later save or approval is rejected.

19.8 Customer change during review

Field review may continue, but approval requires fresh customer comparison.

19.9 New processing result

The existing review becomes stale when a newer candidate is explicitly selected as current.

19.10 Session expiration

After reauthentication, the actual server state determines whether the action already completed.

19.11 Accepted trade-off

A reviewer may occasionally need to reload and repeat a correction.

19.12 Revisit trigger

Consider soft review ownership when real customers report frequent concurrent-review conflicts.

20. Processing-status update strategy

20.1 Periodic polling

Advantages

- simplest
- reliable across Vercel and Render
- easy recovery after network interruption
- works with background tabs
- no long-lived connection infrastructure
- easy demo behavior

Limitations

- repeated requests
- updates are not instantaneous
- must avoid aggressive polling

20.2 Server-sent events

Advantages

- one-directional live updates
- simpler than WebSockets
- efficient for status streams

Limitations

- long-lived connection behavior
- reconnect handling
- platform and proxy considerations
- more operational complexity than Release 1 needs

20.3 WebSockets

Advantages

- bidirectional real-time communication

Limitations

- unnecessary for status updates
- connection management
- scaling and deployment complexity
- weak justification without collaborative editing

20.4 Realtime database subscriptions

Advantages

- rapid updates

- managed infrastructure

Limitations

- frontend coupling to database change structure
- authorization complexity
- business-state leakage
- another real-time mental model

20.5 Recommendation

Use adaptive periodic polling.

20.6 Behavior

- poll frequently while a visible document is Processing
- poll less frequently on Overview or Documents
- pause or reduce polling in an inactive browser tab
- refresh immediately when the tab becomes active
- stop polling final states
- allow manual refresh after connection failure

20.7 Accessibility

Only meaningful state transitions are announced.

Every stage refresh should not produce an accessibility announcement.

20.8 Accepted trade-off

A completion may appear several seconds after it occurs.

20.9 Upgrade path

Revisit server-sent events when:

- active concurrent users grow
- polling materially loads the backend
- live queue dashboards become necessary
- Voice AI or collaborative operations later require lower latency

21. Document and evidence storage architecture

21.1 Original PDFs

- private object storage
- immutable after upload
- no public URL
- authorized staff download
- synthetic sample download under demo policy

- retained with document history

21.2 Temporary processing files

Examples:

- local converted pages
- temporary images
- transient parser output

Retention:

- worker-local only where possible
- removed after stage completion or failure
- never relied on across worker restarts

Render services have ephemeral local filesystems by default, so durable artifacts must not depend on worker-local storage.

21.3 Extracted text

Preserve when required for:

- evidence
- reproducibility
- evaluation
- diagnosis

Avoid duplicating large text unnecessarily.

21.4 OCR output

Preserve the useful textual result and page relationship.

Raw intermediate images should be temporary unless needed for evidence or evaluation.

21.5 Page images

Store only when:

- PDF rendering requires a derived page artifact
- evidence highlighting needs it
- OCR provider workflow requires durable page access

Otherwise, render from the original PDF.

21.6 Evidence excerpts

Store or reproducibly derive:

- excerpt
- page

- source type
- exact location metadata when reliable

21.7 Exact location metadata

Optional.

Absence must be represented explicitly.

21.8 Raw AI responses

Recommendation:

- access restricted to technical operations
- time-limited retention in public demo and development
- longer retention only for approved evaluation documents
- never shown to normal users
- redacted from ordinary logs

21.9 Normalized candidates

Persist immutably per attempt.

21.10 Approved versions

Persist as immutable relational business data, not only as a generated JSON document.

The exact data design belongs in the next specification.

21.11 Signed access

The backend issues short-lived URLs after authorization. Supabase supports signed URLs that expire after a fixed period.

21.12 Browser restrictions

The browser must not:

- list storage contents
- construct arbitrary object paths
- reuse permanent public URLs
- receive service-level storage credentials

21.13 Retention

Release 1 categories:

Permanent business retention

- source PDF
- normalized candidate

- review decisions
- approved versions
- audit events

Limited diagnostic retention

- raw provider responses
- large OCR intermediates

Temporary

- local processing files
- rendered page scratch files
- provider transfer files

21.14 Duplicate detection

Use content identity derived from the file bytes plus contextual checks.

Do not trust file name alone.

21.15 Logs

Document content, policy numbers, names, addresses, VINs, and raw provider responses must not appear in normal logs.

21.16 Provider transmission

Only the minimum required document content should be transmitted.

The Document AI specification must decide whether providers receive:

- full PDF
- selected pages
- extracted text
- page images
- or a combination

22. Authentication, sessions, and authorization

22.1 Option A: Supabase Auth

Benefits

- reduces platform count
- integrates with the preferred Supabase ecosystem
- supports Next.js
- provides signed JWT sessions
- backend can verify identity independently
- supports future security options without implementing authentication from scratch

Supabase Auth uses signed JWT access tokens and provides documented Next.js server-side integration.

Limitations

- coupling to Supabase identity concepts
- careful server-side session handling required
- default session settings may need tightening before real customer use
- backend membership checks remain necessary

22.2 Option B: Separate managed auth provider

Examples include dedicated identity SaaS products.

Benefits

- polished identity features
- potentially stronger enterprise SSO path
- identity separated from database platform

Limitations

- another provider and bill
- additional integration
- no Release 1 enterprise identity need
- still requires backend authorization

22.3 Option C: Custom authentication

Rejected.

It creates unnecessary password, session, recovery, and security responsibility.

22.4 Recommendation

Use Supabase Auth.

22.5 Staff session

- authenticated session issued by Supabase Auth
- frontend maintains session
- FastAPI verifies token authenticity and expiry
- backend resolves staff membership and agency context

22.6 Controlled demo session

Use a dedicated short-lived demo identity recognized by the backend.

It must not be a shared staff credential.

22.7 Protected frontend routes

Frontend checks session for navigation UX.

Backend remains the final access authority.

22.8 One-agency context

Staff membership maps to the single Release 1 agency.

Do not place agency authority only in browser session metadata.

22.9 Staff abilities

All normal authenticated staff users may perform the approved Release 1 workflow.

22.10 Demo restrictions

The backend enforces:

- session-specific synthetic data
- curated samples
- restricted live processing
- simulated email
- prohibited destructive actions
- no real data

22.11 Expired sessions

FastAPI rejects expired authentication.

The frontend reauthenticates and returns to current state.

22.12 Service credentials

Backend and worker use separate server-only credentials where practical.

The worker should receive only the permissions required for processing.

22.13 Document authorization

Every signed read or download requires:

- valid identity
- object relationship
- staff or demo-context match
- current permission

22.14 Audit identity

Business actions record the backend-resolved actor, not an actor name supplied by the browser.

22.15 Revisit trigger

Consider a separate enterprise identity provider when:

- several agencies are supported
 - SSO is contractually required
 - enterprise lifecycle management becomes necessary
-

23. Demo-environment architecture

23.1 Option 1: Shared mutable demo workspace

Benefit

Simplest.

Problem

Visitors interfere with each other and break predictable Loom states.

Rejected.

23.2 Option 2: Per-session disposable database state

Benefit

Real server workflow and isolation.

Problem

Repeated seed data can grow storage and database usage.

23.3 Option 3: Mostly client-side simulation

Benefit

Low backend cost.

Problem

Does not prove real approval, audit, policy versioning, or server rules.

Rejected as the main approach.

23.4 Option 4: Read-only demo

Benefit

Safest.

Problem

Fails to demonstrate corrections, approval, audit, email, and reset.

23.5 Recommendation: practical hybrid

Use:

- immutable shared synthetic templates
- immutable shared sample PDFs
- immutable shared stored extraction templates
- per-session disposable business state
- real backend review, approval, policy-version, audit, and usage behavior
- simulated email
- optional restricted live processing

23.6 Session isolation

Each demo session receives its own:

- customer instances
- document workflow instances
- review state
- approved versions
- audit events
- usage allowance

Large sample files and stored candidate templates may be shared read-only.

23.7 Reset

Reset replaces disposable session state with a fresh seed state.

23.8 Data expiration

Inactive demo sessions expire and are cleaned up automatically.

23.9 Permanent sample protection

Template definitions and sample artifacts cannot be modified by demo users.

23.10 Separation from real data

Public demo should use a separate environment and database project from future customer production.

23.11 Cost consequences

Costs remain controlled because:

- large sample artifacts are shared

- stored processing is default
- disposable records are small
- sessions expire
- live processing is limited

23.12 Provider outage

The complete stored-result golden path remains available.

23.13 Accepted trade-off

Demo-session lifecycle adds architecture beyond a shared account, but it is justified by the product's portfolio purpose.

24. Stored-result replay architecture

24.1 Same workflow requirement

A stored result must create:

- a real document workflow instance
- a real processing attempt
- real stage transitions
- a real candidate review result
- real warning resolution
- real reviewer corrections
- real approval
- real policy versions
- real audit history
- simulated email outcome

24.2 What is simulated

- direct PDF processing computation
- OCR-provider execution
- AI-provider execution
- optional stage timing
- email delivery

24.3 What is real

- authentication
- authorization
- demo restrictions
- attempt lifecycle
- review state
- readiness
- approval
- current-policy replacement
- audit events

- usage classification
- reset

24.4 Stored-result identity

Attempt origin remains:

Stored Result

The interface and audit history display that no live AI processing occurred.

24.5 Shared processing path

Stored normalized candidate artifacts must enter the same Extraction Results boundary used by live processing.

Do not build a separate mock-only review data model.

24.6 Divergence prevention

Every stored sample should be validated against the current normalized candidate contract before public release.

24.7 Revisit trigger

None for Release 1. Stored results remain useful for regression tests even after live providers change.

25. Audit architecture

25.1 Ownership

The Audit module owns append-only business audit history.

Each business module is responsible for supplying accurate event context.

25.2 Business audit events

Include:

- customer created or changed
- upload accepted
- duplicate warning overridden
- attempt started or failed
- OCR used
- result ready
- correction saved
- warning acknowledged
- item added or removed
- approval

- rejection
- policy replaced
- retry or reprocessing requested
- email outcome
- demo reset

25.3 Technical logs

Separate from audit:

- stack traces
- SQL errors
- network timing
- provider request IDs
- worker memory use
- internal retries
- health checks

25.4 Actor identity

Audit events record:

- staff user
- demo user
- system
- background worker action where relevant

25.5 Timestamps

Use one authoritative server time.

Display localization belongs to the frontend.

25.6 Object relationships

Events relate conceptually to:

- customer
- document
- attempt
- review
- approved policy version
- email delivery
- demo session

25.7 Previous and new values

Store only for material changes and minimize unnecessary personal information.

25.8 Append-only expectation

Normal product behavior does not edit or delete audit events.

Corrective events append new facts.

25.9 Same-transaction events

Must be created with:

- customer creation
- saved material correction
- warning acknowledgement
- approval
- rejection
- Current-policy replacement
- explicit contact-address update

25.10 Background events

Processing completion and failure should be committed with the attempt transition.

25.11 Sensitive-data minimization

Avoid copying complete documents, OCR text, or raw provider output into audit history.

26. Usage and cost-control architecture

26.1 Usage categories

Record:

- document submitted
- page count
- direct-text attempt
- OCR pages
- AI attempt
- internal recovery
- Technical Retry
- Reprocessing
- failed attempt
- Stored Result
- email attempt

26.2 Backend enforcement

The backend must enforce:

- 15 MB
- 10 pages
- one active attempt
- Technical Retry limit
- live-processing allowance
- demo-session allowance

- duplicate override confirmation
- processing-disabled state

26.3 Reservation before processing

Before paid work begins:

- confirm limit
- reserve expected allowance
- create attempt and job atomically

This reduces concurrent overspending.

26.4 Finalization

After processing:

- replace estimate with actual provider quantities where available
- record failure usage
- release unused reservation where appropriate

26.5 Cost estimates

Derived from:

- provider-reported usage
- configured rate information
- page counts

When rate information is unavailable, show usage quantities only.

26.6 Approaching limit

A configurable threshold creates an Important operational warning.

26.7 Limit reached

The backend rejects new live work.

Existing:

- documents
- reviews
- approvals
- Stored Results

remain usable.

26.8 Public-demo allowance

May be:

- per session
- per day
- global period allowance

The backend evaluates all applicable limits.

26.9 Owner-controlled limits

Release 1 limits may be deployment configuration rather than a general Settings UI.

26.10 Frontend role

Displays server-calculated allowance.

It does not subtract or authorize usage independently.

27. Email architecture

27.1 Eligibility

The Email module verifies:

- approved policy version
- valid customer email
- staff or demo context
- no conflicting active send
- permitted resend

27.2 Fixed content

Subject and body are controlled by the backend configuration.

The frontend previews the same approved content.

27.3 Recipient

Normal staff delivery uses the customer's stored email.

Demo delivery is simulated or restricted.

27.4 Recommendation

Send email as background work.

27.5 Why

Email is an external dependency and should not keep the approval page waiting.

27.6 Sequence

1. Staff confirms send.
2. Backend creates one send request.
3. UI shows Sending.
4. Worker sends.
5. Email becomes Sent or Failed.
6. Audit records outcome.

27.7 Approval independence

Email failure never rolls back approval.

27.8 Duplicate protection

Use product-level safe submission and provider idempotency.

27.9 Resend behavior

A deliberate resend creates a new send action with duplicate warning.

27.10 Provider outage

Email stays Failed or retryable.

Policy remains Approved.

27.11 Trade-off

Delivery is not instant in the original request, but the product provides clear asynchronous status.

28. Provider-boundary architecture

Release 1 supports one production provider per capability.

28.1 PDF text extraction boundary

Owns

- document-reading request
- page count
- extracted text
- text-quality indicators
- encrypted or malformed result

Isolates

Parser-specific output.

28.2 OCR boundary

Owns

- accepted page input
- OCR text per page
- quality metadata
- provider usage
- normalized failure category

Isolates

Vendor response and confidence format.

28.3 AI structured extraction boundary

Owns

- prepared document context
- supported-field candidate result
- provider usage
- normalized structure result
- raw response retention decision

Isolates

Model identifiers, response format, provider errors, token terminology.

28.4 Email boundary

Owns

- fixed message delivery
- provider message reference
- outcome
- duplicate-send support

28.5 Storage boundary

Owns

- private object creation
- signed read or upload permission
- deletion
- metadata
- provider error translation

28.6 Authentication boundary

Owns

- sign-in session
- identity token
- token verification material
- session termination

Backend business membership remains outside the provider boundary.

28.7 Provider replacement

Replacement should require changing the relevant boundary implementation and evaluation, not changing Review or Policy Versions.

28.8 Avoid universal framework

Do not add:

- runtime provider selection
- fallback provider chains
- shared lowest-common-denominator framework
- provider routing dashboard

28.9 Revisit trigger

Add a second provider only when:

- the first fails benchmark requirements
- a customer contract requires it
- cost or availability justifies maintained redundancy

29. Security and privacy architecture

29.1 Authentication

All staff operations require valid authentication.

29.2 Authorization

The backend checks access for every business object and file.

29.3 Private storage

All insurance PDFs use private storage.

29.4 Encrypted transport

Browser, backend, storage, and providers communicate over encrypted transport.

29.5 Secret management

Secrets remain in managed environment configuration.

29.6 Signed access

PDF access is short-lived and authorized per request.

29.7 Least privilege

- frontend receives no service credentials
- worker receives only needed server privileges
- email credentials stay with backend/worker
- public demo identities cannot access staff data

29.8 Demo separation

- synthetic data only
- separate environment from future production
- no arbitrary uploads
- restricted email
- session expiration

29.9 Database access

Business data is accessed through FastAPI.

Direct browser database access should not be part of the main product path.

29.10 Sensitive logs

Redact:

- names where unnecessary
- addresses
- policy numbers
- VINs
- source text
- provider prompts and responses
- signed URLs
- credentials

29.11 Provider transmission

Document content is sent only to approved configured providers and only when needed.

29.12 Document deletion

Release 1 should support controlled internal deletion for:

- expired demo sessions

- temporary artifacts
- failed abandoned uploads
- future customer-data requests

General staff deletion is not part of the approved UX.

29.13 Audit integrity

Audit is append-only for normal application behavior.

29.14 Rate limiting

Apply to:

- sign-in attempts
- upload authorization
- upload completion
- processing start
- retry
- reprocessing
- signed file access
- email
- demo reset

29.15 Upload abuse

Controlled through:

- authentication
- limits
- content checks
- private storage
- public-demo sample restriction

29.16 Email abuse

Fixed recipient, fixed content, limits, and idempotency.

29.17 Responsible Release 1 claims

The product may claim:

- authenticated access
- private document storage
- encrypted network transport
- human-reviewed AI results
- audit history
- controlled demo isolation
- basic cost controls

29.18 Claims it must not make

- HIPAA compliant
 - SOC 2 certified
 - universally compliant with insurance regulations
 - carrier-verified
 - formally certified for insurance compliance
 - guaranteed secure
 - guaranteed accurate
 - complete regulatory recordkeeping system
-

30. PDF and upload security

30.1 Incorrect extension

Inspect file content rather than trusting `.pdf`.

Mandatory.

30.2 Malformed PDF

Perform a safe parser check before expensive processing.

Mandatory.

30.3 Password protection

Reject and require an unlocked copy.

Mandatory.

30.4 Large content

Enforce 15 MB before acceptance.

Mandatory.

30.5 Page count

Enforce 10 pages before paid processing.

Mandatory.

30.6 Resource-exhaustion risk

Processing must use:

- time limits
- memory limits

- page limits
- controlled rendering resolution
- worker isolation from the web process

Mandatory.

30.7 Embedded scripts and attachments

Do not execute embedded scripts.

Ignore or reject embedded attachments not required for page rendering.

Mandatory.

30.8 Unsafe filenames

Treat user filenames as labels only.

Generate internal object identity separately.

Mandatory.

30.9 Duplicate files

Use content-based duplicate detection.

Mandatory.

30.10 Unsupported content

Detect and fail as Unsupported Document before approval.

Mandatory.

30.11 Malware scanning

Public demo

Curated samples eliminate arbitrary upload risk.

Owner-only development

A full malware-scanning platform is not required.

Before real customer uploads

Add a lightweight malware scan or managed scanning capability before documents enter deeper processing.

30.12 Recommendation

Release 1 public demo baseline:

- strict type validation
- parser safety
- size and page limits
- no script execution
- private storage
- isolated worker
- curated public samples

Future customer-production hardening:

- malware scanning
 - stricter document sanitization
 - security review of PDF dependencies
 - upload quarantine
 - formal retention policy
-

31. Observability and diagnostic architecture

31.1 Minimum toolset

- Vercel deployment and runtime logs
- Render web and worker logs
- Supabase operational visibility
- one hosted error-tracking service
- backend-generated reference identifiers
- basic uptime and health checks
- business usage metrics stored by the application

31.2 Structured application logs

Include:

- reference identifier
- environment
- service role
- operation category
- result
- duration
- attempt identity
- provider category
- retry count

Exclude sensitive document content.

31.3 Error tracking

Capture:

- frontend exceptions
- backend unhandled errors
- worker failures
- approval transaction failure
- storage and provider errors

31.4 Correlation identifiers

One processing attempt should be traceable across:

- web acceptance
- job creation
- worker stage
- provider call
- result publication
- user-facing failure reference

31.5 Business metrics

- processing completion rate
- failure categories
- stage duration
- queue waiting time
- retry rate
- OCR use
- approval failure
- email failure
- usage-limit rejection
- demo reset failure

31.6 Health checks

Web health checks should confirm the service is running without performing expensive dependency work. Render supports configured health checks for web services.

Worker health should be inferred from:

- worker heartbeat
- recent job claims
- stale-job detection
- platform process state

31.7 Alerting

Initial alerts:

- repeated worker failure
- growing stale-job count

- database unavailable
- storage unavailable
- approval transaction failures
- public demo unavailable
- usage unexpectedly high

31.8 Retention

- short retention for verbose development logs
- limited production diagnostic retention
- longer retention for business audit
- no indefinite raw PII logs

31.9 Separation

Audit history

For users and business accountability.

Operational metrics

For product health and cost.

Diagnostic logs

For engineers investigating failures.

32. Failure and recovery architecture

32.1 Frontend hosting

Effect

UI unavailable.

Preserved

All server state and background work.

Recovery

Platform restoration or redeploy.

Demo

Unavailable until frontend returns.

32.2 Backend web service

Effect

No new actions or status queries.

Preserved

Database, files, active worker jobs.

Recovery

Web service restart or rollback.

32.3 Background worker

Effect

Jobs wait or active jobs lose their claim.

Preserved

Job and attempt state.

Recovery

Another worker reclaims expired work safely.

32.4 PostgreSQL

Effect

Business actions and jobs stop.

Preserved

Depends on managed database recovery and backups.

Recovery

Do not continue processing without authoritative state.

32.5 File storage

Effect

New uploads, source viewing, or processing fail.

Preserved

Relational state.

Recovery

Retry storage operation; do not approve without inspectable source.

32.6 PDF extraction

Effect

Attempt may move to OCR when appropriate or fail.

Preserved

Source PDF.

32.7 OCR

Effect

Scanned document cannot become reviewable.

Recovery

Technical Retry if transient; clearer replacement otherwise.

32.8 AI provider

Effect

No candidate result.

Recovery

Limited internal recovery, then Technical Retry.

Demo fallback

Stored results remain available.

32.9 Email provider

Effect

Email Failed.

Preserved

Approval and send request.

Recovery

Resend later.

32.10 Status polling failure**Effect**

Frontend may display stale Processing state.

Preserved

Server processing.

Recovery

Manual refresh, focus refresh, or next polling cycle.

32.11 Audit behavior

Business failure outcomes create appropriate audit events when meaningful.

Pure infrastructure noise remains in technical logs.

32.12 Diagnostic behavior

Every final user-visible failure receives a reference identifier linked to attempt diagnostics.

33. Environment strategy**33.1 Local development****Purpose**

Daily coding and owner testing.

Data

Synthetic only.

Processing

Stored or mocked by default; intentional live calls.

Secrets

Local development configuration.

Database and storage

Local Supabase-compatible environment or isolated development project.

Email

Simulated.

33.2 Automated test environment

Purpose

Repeatable business-rule and workflow verification.

Data

Generated synthetic fixtures.

Processing

No live paid providers.

Database

Disposable or isolated test database.

Storage

Temporary test storage.

33.3 Shared development or preview

Purpose

Review integrated frontend/backend changes before demo deployment.

Data

Synthetic.

Processing

Stored results by default.

Cost

Kept small.

This may use one shared non-production backend rather than one complete backend stack per frontend branch.

33.4 Public demo

Purpose

Portfolio, Loom, interviews, and client walkthroughs.

Data

Synthetic only.

Processing

Stored results default; restricted live samples optional.

Secrets

Separate demo secrets.

Database and storage

Separate from future production.

Email

Simulated or safe test recipient.

Reset

Enabled.

33.5 Future real-customer production

Purpose

Real agency use.

Data

Real personal information.

Processing

Live.

Secrets

Production-only.

Database and storage

Dedicated production environment.

Email

Real fixed acknowledgement.

Reset

Unavailable.

33.6 Minimum environments before coding

- local development
- automated test configuration

33.7 Minimum before public launch

- local development
- automated test
- separate public demo

33.8 Minimum before a real customer

Add a separate production environment.

Do not reuse public demo infrastructure for real customer data.

34. Local development architecture

34.1 Local frontend

Runs against local or explicitly selected development backend.

34.2 Local backend

Runs the same modular-monolith business logic used in deployment.

34.3 Local worker

Runs separately to preserve the web-versus-worker behavior.

34.4 Database

Recommendation:

Use a local Supabase development stack where practical because it preserves PostgreSQL, Auth, and Storage behavior without using production data.

A dedicated remote development project remains an acceptable fallback when local storage or auth fidelity becomes difficult.

34.5 File storage

Ordinary UI development may use:

- local storage emulation
- synthetic sample files
- prepared signed-access behavior

34.6 Stored sample results

Serve as the main frontend and workflow-development input.

34.7 Provider output

Use preserved fixtures for:

- direct text
- OCR
- AI candidates
- warnings
- failure categories

34.8 Intentional live testing

Live providers are invoked only for:

- pipeline evaluation
- regression checks
- provider-specific integration testing
- final demo verification

34.9 Test documents

Maintain a curated synthetic benchmark.

34.10 Email

Use simulated local delivery or a safe provider test destination.

Resend provides test addresses intended to simulate delivery outcomes without sending to real recipients.

34.11 Services not required for ordinary frontend work

- live AI

- live OCR
 - real email delivery
 - always-on Render worker
 - public demo environment
-

35. Deployment and release responsibilities

35.1 Source control

GitHub remains the source of truth.

35.2 Frontend deployment

Vercel builds and deploys the approved frontend commit.

35.3 Backend deployment

Render deploys the FastAPI web runtime.

35.4 Worker deployment

Render deploys the worker from the same compatible source version as the web service.

35.5 Database migrations

Migrations are applied through a controlled release step before new code depends on them.

35.6 Environment variables

Each environment has separate:

- database
- storage
- authentication
- provider
- email
- demo
- observability configuration

35.7 Health checks

Verify:

- frontend loads
- web service responds
- database connection works
- storage authorization works
- worker reports healthy activity

35.8 Release verification

Use a stored-result workflow to verify:

- upload or sample selection
- processing status
- review
- approval
- audit
- email simulation

No paid AI call should be required for every release.

35.9 Rollback

Frontend, web, and worker versions must be rollback-compatible.

A rollback must not reverse database migrations destructively without a separate safe plan.

35.10 Demo preparation

Before public release:

- seed templates validated
- stored results compatible
- demo reset tested
- live allowance configured
- no real data present

35.11 Stored-result preparation

Stored artifacts should be versioned conceptually with the candidate contract and benchmark configuration.

35.12 Inconsistent-deployment risks

New frontend with old backend

May request unsupported behavior.

New web with old worker

May create jobs the worker cannot understand.

New worker with old database

May fail or corrupt attempt interpretation.

35.13 Mitigation

- deploy from compatible release

- worker and web version checks
- backward-compatible changes where possible
- release verification before public promotion

Render supports Git-based deployments and separately deployed service types, making version coordination an application responsibility.

36. Scaling path

Stage 1: Local development and owner testing

Unchanged

- modular monolith
- PostgreSQL-backed jobs
- one web runtime
- one worker runtime
- stored results

Capacity

Minimal.

Do not add

Redis, microservices, autoscaling, provider redundancy.

Stage 2: Public portfolio demo

Unchanged

Core architecture.

May need

- always-on web
- always-on worker
- separate demo database
- automated demo cleanup
- monitoring
- restricted live allowance

Do not add

Multi-tenancy or enterprise infrastructure.

Stage 3: First small agency customer

Unchanged

- modules
- candidate versus approved separation
- immutable versions
- queue pattern
- polling

May need

- production environment
- backups
- stronger retention rules
- malware scanning
- paid database tier
- stronger alerting
- support procedures

Stage 4: Several agencies

Revisit

- multi-tenancy
- workspace ownership
- organization authorization
- data-isolation policies
- invitation and role behavior
- per-agency limits
- separate demo and tenant seeding

Do not automatically add

Microservices.

Stage 5: Higher document volume

May need

- more worker instances
- dedicated Redis or managed task queue
- separate processing capacity
- connection pooling
- stronger job metrics
- provider rate management
- artifact-retention controls

Render allows web and background-worker services to be scaled independently, so more worker capacity does not require changing the frontend or business-module boundaries.

Microservice revisit condition

Consider extracting document processing only when:

- deployment cadence conflicts with the core backend
- different scaling characteristics become materially expensive
- worker dependencies make the web deployment unstable
- a dedicated team owns the processing capability
- isolated security or provider requirements demand it

37. Technology decision summary

Area	Recommendation	Why it fits	Main limitation	Accepted trade-off	Revisit trigger
Frontend	Next.js with TypeScript	Approved stack and strong SaaS routing/rendering fit	Separate frontend/backend deployment	Two application runtimes	Frontend hosting or SSR constraints become material
Frontend hosting	Vercel	First-class Next.js deployment and previews	Additional platform	Platform split	Cost or cross-platform latency becomes excessive
Backend	FastAPI with Pydantic	Clear Python AI integration and validation boundary	Separate runtime from frontend	Two languages	Team productivity materially suffers
Backend hosting	Render web service	Managed long-running Python service	Free service cold starts	Paid demo may be needed	Reliability or cost no longer fits
Worker	Separate Render background worker	Isolates slow work	Separate deployment must stay compatible	Additional service	Throughput needs specialized compute
Database	Supabase PostgreSQL	Relational consistency and existing preference	Database also carries queue load	Simpler platform count	Queue traffic affects business workload

Area	Recommendation	Why it fits	Main limitation	Accepted trade-off	Revisit trigger
Storage	Supabase private Storage	Private objects and signed access	Provider coupling	One integrated platform	Storage volume, region, or compliance changes
Authentication	Supabase Auth	Fewer providers and JWT-based backend verification	Supabase coupling	Simpler Release 1 identity	Enterprise SSO or complex lifecycle required
Background jobs	PostgreSQL-backed durable queue	No extra stateful service	Database polling/load	Lower cost and complexity	Sustained job volume
Status updates	Adaptive polling	Simple and reliable	Several-second delay	No realtime infrastructure	Polling load or latency becomes material
Email	Resend through background worker	Narrow transactional workflow and idempotency support	External dependency	Email is eventually consistent	Delivery requirements expand
Error tracking	One hosted error tracker	Cross-runtime visibility	Additional service	Small operational dependency	Platform-native tools become sufficient or privacy requires self-hosting
Operational logs	Vercel, Render, and Supabase platform logs plus structured application logs	Lowest additional cost	Logs distributed across platforms	Three places to inspect	Operational burden becomes excessive

38. Technical Decision Records

TDR-01: Modular Monolith versus Microservices

1. **Status:** Proposed for approval

2. **Context:** Release 1 has several business modules but one small product and one development owner.
3. **Decision:** Use one modular FastAPI backend codebase with separate web and worker runtimes.
4. **Alternatives:** Microservices; single unstructured monolith.
5. **Why preferred:** Maintains boundaries without distributed-system overhead.
6. **Trade-offs:** Modules are not independently deployable.
7. **Consequences:** Internal ownership must be enforced through design reviews.
8. **Risks:** Hidden coupling inside one codebase.
9. **Revisit trigger:** Independent teams, materially different deployment needs, or high processing scale.
10. **Effect on implementation:** Codex tasks must identify the owning module for every business change.

TDR-02: Vercel, Render, and Supabase deployment split

1. **Status:** Proposed for approval
2. **Context:** Preferred stack uses managed platforms optimized for different responsibilities.
3. **Decision:** Keep Vercel frontend, Render web/worker, Supabase data/Auth/Storage.
4. **Alternatives:** One consolidated server; all services on one cloud provider.
5. **Why preferred:** Minimizes infrastructure administration.
6. **Trade-offs:** Three deployment consoles, secrets, and outage boundaries.
7. **Consequences:** Cross-platform configuration and monitoring are required.
8. **Risks:** Operational fragmentation.
9. **Revisit trigger:** Cost or latency exceeds the value of managed separation.
10. **Effect on implementation:** No durable local files and no direct frontend business-data authority.

TDR-03: Separate Background Worker

1. **Status:** Proposed for approval
2. **Context:** OCR and AI processing are slow and variable.
3. **Decision:** Run processing in a separate worker runtime.
4. **Alternatives:** Web-request processing; serverless task per stage.
5. **Why preferred:** Protects web responsiveness and supports durable jobs.
6. **Trade-offs:** Separate deployment and worker health monitoring.
7. **Consequences:** Work must be represented durably before execution.
8. **Risks:** Web and worker version mismatch.
9. **Revisit trigger:** Managed workflows become justified by pipeline complexity.
10. **Effect on implementation:** Every long task requires attempt and job state.

TDR-04: PostgreSQL-backed Background Queue

1. **Status:** Proposed for approval
2. **Context:** Low initial volume and limited budget.
3. **Decision:** Use PostgreSQL-backed durable jobs.
4. **Alternatives:** Redis-backed jobs; managed workflow service.
5. **Why preferred:** Avoids another stateful paid service.
6. **Trade-offs:** Queue load shares the primary database.
7. **Consequences:** Polling, claims, leases, and deduplication must be designed carefully.
8. **Risks:** Database contention at higher volume.
9. **Revisit trigger:** Sustained queue volume or many worker instances.
10. **Effect on implementation:** Job creation can be committed with attempt and usage state.

TDR-05: Processing Status through Polling

1. **Status:** Proposed for approval
2. **Context:** Status changes occur over seconds or minutes, not milliseconds.
3. **Decision:** Use adaptive polling.
4. **Alternatives:** Server-sent events, WebSockets, realtime database subscriptions.
5. **Why preferred:** Lowest complexity and broad deployment compatibility.
6. **Trade-offs:** Small update delay and repeated requests.
7. **Consequences:** Polling frequency must adapt to page and tab activity.
8. **Risks:** Excessive polling if implemented carelessly.
9. **Revisit trigger:** Larger concurrent user count or lower-latency requirement.
10. **Effect on implementation:** Status resources must be cheap and cache-aware.

TDR-06: Immutable Approved Policy Versions

1. **Status:** Proposed for approval
2. **Context:** Approved information must be explainable and historically preserved.
3. **Decision:** Every approval creates an immutable policy version.
4. **Alternatives:** One mutable policy record.
5. **Why preferred:** Provides traceability, correction history, and safe replacement.
6. **Trade-offs:** More historical data and version-aware UX.
7. **Consequences:** Corrections create new versions.
8. **Risks:** Current-versus-historical rules become inconsistent if duplicated.
9. **Revisit trigger:** None expected; this remains a core principle.
10. **Effect on implementation:** Approved records cannot be edited in place.

TDR-07: Optimistic Concurrency versus Hard Locking

1. **Status:** Proposed for approval
2. **Context:** Several users may open the same review, but assignments are excluded.
3. **Decision:** Use optimistic freshness checks.
4. **Alternatives:** Last-write-wins; hard locks.
5. **Why preferred:** Prevents stale changes without lock recovery.
6. **Trade-offs:** Users may occasionally need to reload.
7. **Consequences:** Every save and final action carries freshness context.
8. **Risks:** Poor stale-error UX could frustrate reviewers.
9. **Revisit trigger:** Frequent concurrency conflicts at real agencies.
10. **Effect on implementation:** Review and customer state require version identities.

TDR-08: Private Document Storage

1. **Status:** Proposed for approval
2. **Context:** PDFs contain personal insurance information.
3. **Decision:** Use private Storage and short-lived signed access.
4. **Alternatives:** Public URLs; proxy every file byte through FastAPI.
5. **Why preferred:** Secure access without unnecessary backend bandwidth.
6. **Trade-offs:** Signed-access lifecycle must be managed.
7. **Consequences:** Browser cannot list or guess objects.
8. **Risks:** Overly long signed URLs or leaked server credentials.
9. **Revisit trigger:** Stronger customer-specific storage requirements.
10. **Effect on implementation:** Backend authorizes every read and upload permission.

TDR-09: Supabase Authentication Direction

1. **Status:** Proposed for approval
2. **Context:** One agency, simple staff access, Supabase already selected.
3. **Decision:** Use Supabase Auth with backend verification and membership checks.
4. **Alternatives:** Separate identity SaaS; custom authentication.
5. **Why preferred:** Reduces providers and custom security work.
6. **Trade-offs:** Increased Supabase coupling.
7. **Consequences:** Browser session and backend authorization remain separate responsibilities.
8. **Risks:** Misconfigured token validation or overly broad database policies.
9. **Revisit trigger:** Enterprise SSO or multi-agency identity requirements.
10. **Effect on implementation:** Service credentials remain server-only.

TDR-10: Demo-Session Isolation

1. **Status:** Proposed for approval
2. **Context:** Shared mutable demo data is unreliable.
3. **Decision:** Use disposable per-session business state with shared immutable templates.
4. **Alternatives:** Shared demo workspace; client simulation; read-only demo.
5. **Why preferred:** Real workflows without visitor interference.
6. **Trade-offs:** Session seeding, expiration, and cleanup.
7. **Consequences:** Demo state must be partitioned from staff data.
8. **Risks:** Unbounded abandoned-session growth.
9. **Revisit trigger:** Demo traffic makes per-session state expensive.
10. **Effect on implementation:** Every demo business object is associated with a session context.

TDR-11: Stored Results as Default Public Demo Mode

1. **Status:** Proposed for approval
2. **Context:** Live providers are variable and cost money.
3. **Decision:** Stored Result is the default; curated live runs are optional.
4. **Alternatives:** Live-only demo; fully simulated frontend.
5. **Why preferred:** Reliable, transparent, and low-cost.
6. **Trade-offs:** Some visitors may perceive it as less spontaneous.
7. **Consequences:** Stored results must remain contract-compatible with live results.
8. **Risks:** Replay path diverges from production.
9. **Revisit trigger:** Live processing becomes cheap and consistently reliable.
10. **Effect on implementation:** Replay enters the same candidate and review pipeline.

TDR-12: One Provider per Capability

1. **Status:** Proposed for approval
2. **Context:** Provider flexibility is useful, but maintaining several integrations is expensive.
3. **Decision:** One production provider per OCR, AI, email, Auth, and Storage capability, behind a clean boundary.
4. **Alternatives:** Direct provider calls throughout modules; universal multi-provider framework.
5. **Why preferred:** Replaceability without premature routing.
6. **Trade-offs:** No automatic fallback.
7. **Consequences:** Provider outages create clear failure or Stored Result fallback.
8. **Risks:** Single-provider dependency.
9. **Revisit trigger:** Benchmark failure, customer requirement, or justified redundancy.
10. **Effect on implementation:** Provider-specific fields remain within boundary adapters.

TDR-13: Audit Events versus Technical Logs

1. **Status:** Proposed for approval
 2. **Context:** Staff need understandable history; engineers need diagnostics.
 3. **Decision:** Keep append-only business audit separate from diagnostic logs.
 4. **Alternatives:** One combined event stream.
 5. **Why preferred:** Prevents technical noise and sensitive leakage in the user interface.
 6. **Trade-offs:** Some actions produce both an audit event and a log.
 7. **Consequences:** Business actions define audit semantics explicitly.
 8. **Risks:** Audit may be omitted if not transactionally integrated.
 9. **Revisit trigger:** Formal compliance later requires stronger audit infrastructure.
 10. **Effect on implementation:** Every critical business operation includes an audit requirement.
-

39. Architecture acceptance criteria

39.1 Deployment

- Frontend, web, and worker can be deployed independently.
- Web and worker report compatible release versions.
- Worker restart does not lose durable pending work.
- No durable business artifact depends on Render's local filesystem.
- A stored-result golden path verifies every release without live AI calls.

39.2 Authentication

- Unauthenticated requests cannot access customers, documents, reviews, or signed files.
- Backend independently verifies user identity.
- Demo identity cannot access staff data.
- Expired sessions cannot complete approval.
- Audit actor identity comes from verified backend context.

39.3 Document storage

- Original PDFs are private.
- Browser access requires a short-lived authorized link.
- Storage object paths are not accepted as proof of authorization.
- Temporary worker files are removed.
- Public demo contains synthetic files only.

39.4 Background processing

- Upload acceptance creates durable processing work before showing active Processing.
- Browser closure does not cancel work.
- Only one worker owns a job at a time.
- Abandoned claims can be recovered safely.
- Long-running attempts become visible as delayed or failed.

39.5 Processing attempts

- Initial, retry, reprocessing, and Stored Result attempts remain distinguishable.
- Previous attempts are never overwritten.

- One document can have several attempts.
- Only one attempt is active at a time.
- Only one candidate is current for review.

39.6 Review state

- Saved corrections remain separate from candidate data.
- Mark Not Present is distinguishable from empty.
- Added and removed list items remain traceable.
- Backend calculates readiness.
- Unsaved browser edits cannot become approved.
- Stale saves are rejected.

39.7 Approval

- Approval rechecks readiness and freshness.
- Approval either completes all required effects or none.
- Repeated approval creates one policy version.
- Approval never depends solely on frontend state.
- Explicit address update is applied only when selected.

39.8 Policy versions

- Approved versions cannot be edited in place.
- Replacing Current preserves the previous version.
- An older document can be approved Historical.
- Correction Review creates a new version.
- Failed reprocessing does not affect Current.

39.9 Retries

- No more than two user-triggered Technical Retries are accepted.
- Retry eligibility is enforced by the backend.
- Retry preserves previous attempts and approved data.
- Duplicate retry submission returns the active attempt.

39.10 Duplicate safety

- Repeated upload completion does not create several documents.
- Repeated customer creation submission does not create duplicate outcomes.
- Repeated approval, rejection, email, or reset remains safe.

39.11 Concurrency

- Two users may view the same review.
- One user cannot overwrite a newer saved review.
- One user cannot approve after another user rejects.
- Customer changes trigger approval reconfirmation.
- A newer processing result makes an older final action stale.

39.12 Status updates

- Processing status becomes visible without page reload.

- Polling stops for final states.
- Temporary polling failure does not change server state.
- Background completion produces at most one user notification.

39.13 Audit

- Approval and its audit event cannot become inconsistent.
- Material corrections include previous and new values.
- Technical stack traces never appear in user audit.
- Audit events identify actor and server timestamp.
- Normal users cannot edit audit events.

39.14 Usage limits

- Limits are checked before paid work.
- Concurrent submissions cannot bypass the limit.
- Failed attempts remain counted appropriately.
- Stored Results do not appear as live provider usage.
- Live Processing Disabled remains enforced when frontend controls are bypassed.

39.15 Demo mode

- Each disposable demo session cannot view another session's mutable state.
- Templates cannot be permanently modified.
- Reset returns the session to a known state.
- Provider outage does not break the stored golden path.
- Arbitrary public uploads remain blocked.

39.16 Provider outages

- OCR or AI outage cannot create an approved candidate.
- Email outage cannot reverse approval.
- Storage outage prevents source-dependent approval.
- Final failures have diagnostic references.

39.17 Security

- Service credentials never reach the browser.
- Signed URLs expire.
- File content is validated independently of extension.
- Size and page limits are server-enforced.
- Sensitive content is redacted from logs.
- Public demo and future production use different data environments.

39.18 Observability

- One attempt can be traced across web, worker, and provider stages.
- Stale jobs can be identified.
- Approval transaction failures are captured.
- Usage-limit rejections are measurable.
- Health status does not require paid provider calls.

39.19 Local development

- Frontend work can proceed without live OCR, AI, or email.
- Local worker behavior can be tested.
- Stored fixtures use the same normalized candidate boundary.
- No real customer data is needed.

40. Architecture risks

Risk	Likelihood	Impact	Mitigation	Accepted residual risk	Blocks implementation
Too many hosting platforms	Medium	Medium	Clear ownership, documented secrets, unified release checklist	Some operational context switching	No
Render cold starts or worker availability	High on free capacity	High for demos	Use stored results; move public demo to always-on capacity	Platform outage remains possible	Public launch only
Background-job duplication	Medium	High	Durable claims, one active attempt, idempotent stage completion	Rare recovery conflicts	Yes
Stale frontend state	Medium	High	Optimistic concurrency and polling refresh	User may repeat work	Yes
Document-access leakage	Low if designed correctly	Critical	Private storage, signed access, backend authorization	Signed link may remain valid briefly after sharing	Yes
Demo-session data growth	Medium	Medium	Expiration, cleanup, shared immutable artifacts	Temporary growth between cleanup runs	Public launch

Risk	Likelihood	Impact	Mitigation	Accepted residual risk	Blocks implementation
Stored replay diverges from live	Medium	High	Same normalized candidate boundary and contract validation	Stored samples may lag new provider behavior	Yes
Provider abstraction over-engineering	Medium	Medium	One provider per capability, narrow boundaries	Later replacement still requires work	No
Audit and business state inconsistent	Low with transactions	Critical	Audit in same transaction for critical actions	Background diagnostic events may arrive separately	Yes
Raw AI/OCR data exposes PII	Medium	High	Restricted retention, redacted logs, limited access	Authorized technical staff may still access retained artifacts	Yes
Web and worker incompatible versions	Medium	High	Same release identity, compatibility checks, coordinated deploy	Brief mixed-version windows may occur	Yes
PostgreSQL queue bottleneck	Low in Release 1	Medium	Slow polling, indexed claims, queue metrics	Higher future volume requires migration	No
PDF viewer or signed file access slow	Medium	Medium	Direct signed download, regional alignment, page caching	Large scans may remain slower	No
Cost controls only in frontend	Medium if poorly designed	High	Backend reservation and enforcement	Provider billing may differ from estimates	Yes

Risk	Likelihood	Impact	Mitigation	Accepted residual risk	Blocks implementation
Supabase becomes broad dependency	Medium	Medium	Maintain clear Auth, Storage, and DB boundaries	Migration would still be substantial	No
Worker processes malicious PDF	Low with curated demo; higher with real users	High	strict validation, isolation, resource limits; malware scanning before customer launch	Unknown parser vulnerabilities remain	Customer launch

41. Architecture scope-creep findings

Microservices

Useful for independent teams and scale. Not justified for one product and owner.

Revisit when module deployment and ownership genuinely diverge.

Kubernetes

Useful for complex container orchestration. Managed platforms already solve Release 1 deployment.

Revisit only at substantial scale or enterprise cloud requirement.

Message streaming

Useful for high-volume event distribution. Release 1 has durable jobs and audit events, not a streaming workload.

Multiple databases

May isolate workloads later. One PostgreSQL database provides stronger Release 1 consistency.

Multi-region hosting

Useful for global availability. Not justified for a portfolio demo or first small agency.

Active-active failover

Operationally expensive and unnecessary.

Multi-provider AI routing

Attractive for resilience and cost optimization. It multiplies evaluation and failure behavior.

Revisit after one provider is benchmarked and real volume exists.

WebSocket infrastructure

Useful for bidirectional real-time collaboration. Polling is sufficient.

Service mesh

Solves microservice communication problems that do not exist.

Enterprise identity

Useful for SSO and lifecycle management. One-agency staff access does not require it.

Advanced secrets platform

Managed platform secrets are sufficient initially.

Revisit with several environments, teams, or formal security requirements.

Data warehouse

Advanced analytics are excluded.

Vector database

No Release 1 semantic-search or document-chat use case.

Event sourcing

Immutable policy versions and audit do not require rebuilding all state from events.

CQRS

Read and write scale do not justify separate models.

Generalized workflow engine

The pipeline is narrow and linear.

Complex feature-flag platform

Deployment configuration and a small number of controlled environment options are sufficient.

High-volume malware-scanning platform

Not needed for curated public samples.

A lightweight scan becomes appropriate before real external uploads.

Enterprise compliance tooling

Formal certification is not a Release 1 claim.

42. Open architecture decisions

Decision 1: Approve the three-platform topology

Recommendation

Vercel + Render + Supabase.

Alternative

One consolidated server.

Cost impact

Managed split may cost slightly more than one small server after free tiers.

Complexity impact

More platform configuration, less infrastructure maintenance.

Reliability impact

Independent failure boundaries, managed services.

Effect on later documents

Data, API, deployment, and security specifications assume this split.

Effect on Codex

Codex must respect browser, web, worker, database, and storage boundaries.

Decision 2: Approve PostgreSQL-backed jobs

Recommendation

Use the primary PostgreSQL database for Release 1 job state.

Alternative

Add Redis immediately.

Cost impact

Avoids another paid service.

Complexity impact

Requires careful job claims but fewer platforms.

Reliability impact

Strong job/business transaction consistency; shared database risk.

Later documents

The data specification must represent job ownership and attempts conceptually.

Codex effect

Codex must not run long processing inside web requests.

Decision 3: Approve Supabase Auth**Recommendation**

Use Supabase Auth with FastAPI verification.

Alternative

Separate managed identity provider.

Cost impact

Lower platform count.

Complexity impact

Simpler Release 1 integration.

Reliability impact

Authentication shares Supabase dependency.

Later documents

API and security specifications use Supabase-issued identity.

Codex effect

Codex must not treat frontend session claims as business authorization.

Decision 4: Approve direct-to-private-storage upload**Recommendation**

Backend-authorized direct browser transfer to private Storage.

Alternative

Proxy the complete file through FastAPI.

Cost impact

Reduces backend bandwidth and compute.

Complexity impact

Adds short-lived upload authorization.

Reliability impact

Large transfers do not occupy web workers.

Later documents

Upload API contract must support authorization and completion.

Codex effect

Codex must prevent arbitrary storage path selection.

Decision 5: Approve limited raw-provider retention**Recommendation**

Retain raw AI and OCR provider responses for a short configurable diagnostic period, with longer retention only for curated evaluation samples.

Alternative

Retain forever or delete immediately.

Cost impact

Moderate storage savings compared with indefinite retention.

Complexity impact

Requires cleanup rules.

Reliability impact

Enough evidence for diagnosis without unlimited PII accumulation.

Later documents

Document AI specification must identify which raw artifacts are required for evaluation.

Codex effect

Codex must not place raw output in ordinary logs or audit.

Decision 6: Approve paid always-on demo capacity when launching publicly**Recommendation**

Develop on low-cost capacity, but use always-on backend and worker hosting once the public demo becomes part of active Upwork marketing.

Alternative

Accept cold starts.

Cost impact

Adds recurring demo hosting.

Complexity impact

No major code complexity.

Reliability impact

Materially better Loom and live-demo experience.

Later documents

Deployment specification identifies a launch-readiness gate.

Codex effect

None beyond health and startup behavior.

43. Human supervision checkpoints

43.1 Deployment topology

You should explain

Why frontend, web backend, worker, database, and storage are separated.

Trade-off approved

Three managed platforms instead of one server.

Risk if misunderstood

Business logic may leak into the frontend or worker deployment may be omitted.

Evidence to inspect

Deployment diagram, environment separation, actual service list, network paths.

43.2 Authentication

You should explain

Authentication proves identity; FastAPI authorization decides access.

Trade-off

Supabase coupling for reduced platform count.

Risk

A valid login could be mistaken for permission to access every record.

Evidence

Backend token verification, membership checks, signed-file authorization.

43.3 Background-job strategy

You should explain

Why long work cannot run inside an upload request.

Trade-off

PostgreSQL handles queue state.

Risk

Duplicate jobs, timeouts, or lost work.

Evidence

Durable job creation, worker claim, restart recovery, delayed-job detection.

43.4 Document storage

You should explain

Why PDFs are private and accessed through short-lived permission.

Trade-off

More access-control logic than public URLs.

Risk

Sensitive document leakage.

Evidence

Storage policy, signed URL expiry, browser credential inspection.

43.5 Candidate versus approved separation

You should explain

Why extracted AI values cannot directly update the customer policy.

Trade-off

More data layers and review state.

Risk

AI errors become business records.

Evidence

Candidate, review correction, and approved-version displays and persistence behavior.

43.6 Policy versioning

You should explain

Why approval creates a version rather than updating one policy row.

Trade-off

Historical storage and version-aware UX.

Risk

Silent loss of prior approved information.

Evidence

Replacement workflow and preserved history.

43.7 Retry and idempotency

You should explain

Why repeating the same action must not repeat the business outcome.

Trade-off

Every critical action needs duplicate-safety design.

Risk

Repeated charges, policies, emails, or audit events.

Evidence

Double-submission tests and active-attempt protection.

43.8 Concurrency

You should explain

Why optimistic concurrency is preferred over hard locks.

Trade-off

Some users must reload stale work.

Risk

Last-write-wins overwrites reviewer decisions.

Evidence

Two-browser review and approval scenario.

43.9 Demo-session architecture

You should explain

What is shared, what is disposable, and what is genuinely executed.

Trade-off

Session cleanup complexity for demo reliability.

Risk

Visitors damage shared data or think stored results are live.

Evidence

Separate sessions, reset, Stored Result labels.

43.10 Usage enforcement

You should explain

Why limits are enforced before job creation by the backend.

Trade-off

Usage reservation logic.

Risk

Frontend bypass causes uncontrolled cost.

Evidence

Direct backend submission at limit and concurrent-attempt tests.

43.11 Audit ownership

You should explain

Difference between user audit and technical logs.

Trade-off

Critical business actions create both business and operational records.

Risk

Audit becomes inconsistent or leaks PII.

Evidence

Approval transaction and audit result.

43.12 Security boundaries

You should explain

Which credentials reach browser, backend, and worker.

Trade-off

Managed secret configuration across platforms.

Risk

Service credentials or files become public.

Evidence

Environment audit, browser network inspection, log redaction.

43.13 Observability

You should explain

How one failed attempt is followed from upload through worker and provider failure.

Trade-off

Structured logging and reference identifiers add engineering work.

Risk

Failures cannot be diagnosed during demos or interviews.

Evidence

Failure Detail reference linked to technical logs.

44. Architecture decision log

Decision	Reason	Main alternative	Accepted trade-off	Status	Revisit condition
Modular monolith	Clear boundaries without distributed complexity	Microservices	Modules not independently deployed	Proposed	Independent teams or scale

Decision	Reason	Main alternative	Accepted trade-off	Status	Revisit condition
Vercel + Render + Supabase	Managed strengths and preferred stack	One server	Three platforms	Proposed	Cost or latency becomes excessive
Separate worker	Slow work must leave request path	Web-process jobs	Separate deployment	Proposed	Managed workflows become justified
PostgreSQL queue	Lowest cost and strong consistency	Redis	Shared DB load	Proposed	Sustained high job volume
Adaptive polling	Simplest reliable status updates	SSE/ WebSockets	Small update delay	Proposed	Polling load or latency requirement
Immutable policy versions	Auditability and safe corrections	Mutable policy	More historical data	Proposed	No planned reversal
Optimistic concurrency	Prevent stale writes without locks	Hard locking	Occasional reload	Proposed	Frequent team conflicts
Private signed file access	Protect insurance documents	Public URLs	Signed-access complexity	Proposed	Stronger enterprise controls
Supabase Auth	Fewer platforms	Separate identity SaaS	Supabase coupling	Proposed	Enterprise SSO
Isolated demo sessions	Predictable demos	Shared demo account	Cleanup work	Proposed	High demo traffic
Stored Result default	Reliability and cost control	Live-only demo	Less spontaneity	Proposed	Live processing becomes consistently cheap
One provider per capability	Avoid over-engineering	Provider routing	No automatic fallback	Proposed	Contract or benchmark need
Separate audit and logs	User clarity and privacy	Combined event stream	Duplicate recording effort	Proposed	Formal compliance tooling

45. Recommended next planning document

The next document should be:

Release 1 Document AI Pipeline and Evaluation Specification

Architecture is now sufficiently defined to decide how one supported declaration page becomes a high-quality reviewable candidate.

The next document should define:

- supported synthetic and permitted evaluation benchmark
- clean PDF and scanned PDF categories
- direct PDF text-extraction behavior
- OCR-decision criteria
- rotated-page handling
- document-type classification
- one-policy-per-file detection
- structured field-extraction contract
- field normalization
- source-page evidence
- exact-location evidence where available
- missing and conflicting value handling
- validation rules
- uncertainty signals
- raw-response retention
- extraction and prompt versioning
- provider comparison
- model comparison
- OCR-provider comparison
- cost evaluation
- latency evaluation
- clean-PDF quality target
- scanned-PDF quality target
- critical-field silent-error target
- repeatability testing
- regression benchmark
- failure corpus
- stored-result generation
- use of reviewer corrections as evaluation evidence
- rules for changing model, prompt, or extraction configuration

It must not yet define:

- exact database tables
- exact public API endpoints
- final production code
- repository structure

- implementation tickets
- Codex prompts
- detailed UI design
- Voice AI
- several production AI providers
- automatic model training from reviewer corrections

This sequence is important because the data specification and API contracts should be built around a proven Document AI result model, not around an assumed extraction format.